

BCA-402 Computer Graphics and Multimedia

Table of Contents

BCA-402: COMPUTER GRAPHICS AND MULTIMEDIA

Complete Exam Handbook — B.R.A. Bihar University (BRABU), Muzaffarpur

A Professional, Exam-Oriented Reference for BCA Students

Covers: Unit I – VI (Full Syllabus) | Definitions | ASCII Diagrams | Solved Numericals | Viva Questions | MCQs | PYQs | Revision Notes

HOW TO USE THIS HANDBOOK

1. Read theory first, then the solved numericals.
2. Every algorithm has: Theory, Formula, Step-by-step Algorithm, Flowchart, Solved Example, Advantages, Disadvantages, Applications.
3. Practice every transformation, clipping, and curve numerical by hand — Units III, IV, and V carry the highest numerical weightage.
4. Use the Memory Trick boxes for quick recall in exams.
5. Revise the Last Minute Revision Sheet at the end, one day before the exam.

BCA-402: COMPUTER GRAPHICS AND MULTIMEDIA

Complete Exam Notes — B.R.A. Bihar University (BRABU), Muzaffarpur

UNIT I — INTRODUCTION TO COMPUTER GRAPHICS

1.1 Introduction to Computer Graphics

What is Computer Graphics?

Definition: Computer Graphics is the branch of computer science that deals with **generating, manipulating, and displaying pictures/images** with the help of a computer. It includes creation, storage, and manipulation of images and pictures using a computer.

Zero-Level Explanation: Think about it this way — before computers, if you wanted to draw a picture, you needed paper, pencil, and your hand. Computer Graphics is like giving the computer a “digital pencil and paper” — the computer becomes the artist. Every image you see on your phone, TV, or laptop screen — icons, videos, games, movies with special effects — all of these are created using the principles of Computer Graphics.

Formal Definition: Computer Graphics can be defined as “a field of computer science that studies methods for digitally synthesizing and manipulating visual content through the use of computers.”

Why do we study Computer Graphics?

- To understand how images are created, stored, and displayed on a screen.
- To learn how movies, games, and animations are made.
- To understand how CAD (Computer-Aided Design) software works for engineering drawings.
- To learn how data can be represented visually (charts, graphs) for better understanding.

Basic Elements Used in Computer Graphics

Element	Meaning
Pixel (Picture Element)	Smallest addressable point on a screen; a screen is made of thousands/millions of pixels
Resolution	Total number of pixels on the screen (e.g., 1920×1080)
Frame Buffer	A part of memory that stores the pixel/image data before it is displayed
Coordinate System	A system (x, y) or (x, y, z) used to define positions of points/objects on screen

Memory Trick: “PRFC” → Pixel, Resolution, Frame Buffer, Coordinate system — the four basic building blocks of any graphics system.

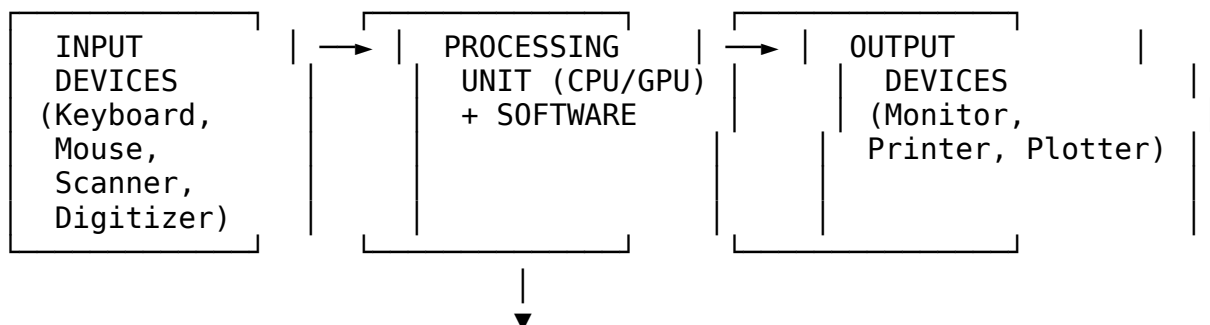
1.2 Computer Graphics System

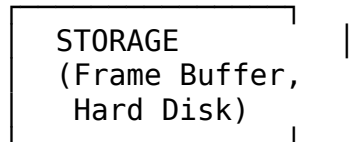
Definition

A **Computer Graphics System** is a computer system that consists of both **hardware** and **software** components which work together to create, manipulate, and display graphical images.

Components of a Computer Graphics System

A complete Computer Graphics System has the following **five major components**:





1. Input Devices

Devices used to give commands/data/drawings to the computer. - **Keyboard** – used to enter text, commands, coordinate values. - **Mouse** – used to point, select, draw, and drag objects on screen. - **Digitizer/Graphics Tablet** – a flat surface used with a stylus pen to trace/draw drawings directly into the computer. - **Scanner** – converts a physical picture/document into a digital image. - **Light Pen** – a pen-like device with a light-sensitive tip, used to draw directly on the CRT screen (older technology). - **Joystick** – used mainly in gaming and simulation for interactive control.

2. Output Devices

Devices used to display or print the graphical output. - **Monitor/VDU (Visual Display Unit)** – the most common output device (CRT, LCD, LED screens). - **Printer** – produces a hard copy (paper output) of the graphic — includes Dot Matrix, Inkjet, Laser printers. - **Plotter** – used for high-precision line drawings, mainly in engineering/architecture (CAD drawings, maps).

3. Processing Unit (Hardware)

- **CPU (Central Processing Unit)** – executes the general program instructions.
- **GPU (Graphics Processing Unit)** – a specialized processor designed to handle graphics computations (rendering, shading, transformations) much faster than a CPU alone.
- **Display Controller/Video Controller** – hardware that manages what is shown on the display, reading from the frame buffer and refreshing the screen.

4. Storage

- **Frame Buffer (Refresh Buffer):** A special memory area that stores the color/intensity value of every pixel on the screen. The image on screen is a direct representation of what is stored in the frame buffer.
- **Secondary Storage:** Hard disks, SSDs used to permanently save images, graphics files, and software.

5. Software

- **Graphics Software/Application Software:** Programs used to create and manipulate graphics — e.g., AutoCAD, Photoshop, CorelDRAW, Blender, 3ds Max.
- **Graphics Library/API (Application Programming Interface):** A set of pre-written functions that programmers use to create graphics — e.g., OpenGL, DirectX.

Summary Table: Computer Graphics System Components

Component	Examples	Purpose
Input Devices	Keyboard, Mouse, Scanner, Digitizer	Give input/commands

Component	Examples	Purpose
Processing Unit	CPU, GPU	Process graphics computations
Storage	Frame Buffer, Hard Disk	Store pixel/image data
Output Devices	Monitor, Printer, Plotter	Display/print the result
Software	AutoCAD, OpenGL, Photoshop	Create/manipulate graphics

Exam Tip: “Explain the components of a Computer Graphics System” is a **very frequently asked 5/10 mark question** at BRABU — always draw the block diagram (Input → Processing → Output, with Storage) and explain each component with 2 examples.

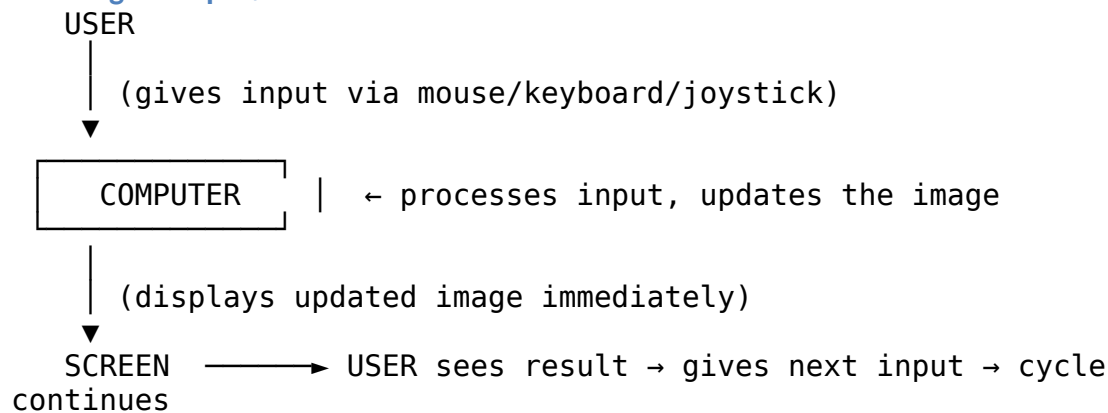
1.3 Interactive Graphics

Definition

Interactive Computer Graphics is a type of computer graphics system in which the **user can directly interact/communicate with the computer in real-time** — the user can control, modify, and manipulate the image while it is being displayed, and the computer responds immediately to the user’s actions.

Zero-Level Explanation: Imagine playing a video game — when you press a button, the character on the screen moves *immediately*. This immediate two-way communication between you (the user) and the computer is called **Interactive Graphics**. You give an input (through mouse/keyboard/joystick), the computer processes it, and you see the result on the screen right away — and you can keep changing your input based on what you see.

Working Principle / How It Works



This continuous loop of **Input → Process → Output → feedback to user** happening in **real time** is the essence of interactive graphics.

Key Characteristics of Interactive Graphics

1. **Two-way communication** between user and computer.
2. **Real-time response** — minimal or no delay between user action and screen update.
3. User has **full control** to modify, move, resize, or edit graphical objects.

4. Uses **input devices** like mouse, keyboard, joystick, touch screen extensively.

Examples of Interactive Graphics

- Video games (e.g., racing games, where the car moves as you steer)
- CAD software (AutoCAD) — engineer draws/edits a design and sees changes instantly
- Paint/drawing software (MS Paint, Photoshop) — user draws and sees the stroke immediately
- Flight simulators
- Touchscreen mobile phone interfaces
- Microsoft PowerPoint (while creating/editing slides)

Advantages of Interactive Graphics

- User has **direct control** over the graphic content.
- **Faster and easier** to make corrections/changes (no need to redo everything).
- Provides a much better and engaging **user experience**.
- Very useful for **design, simulation, and training** purposes (e.g., pilot training simulators).

Disadvantages of Interactive Graphics

- Requires **more powerful hardware** (fast CPU/GPU) to give real-time response.
- **More expensive** to develop and implement.
- Complex programming is required to handle real-time input processing.

Applications of Interactive Graphics

- Video game development
- Computer-Aided Design (CAD/CAM)
- Flight and driving simulators
- Medical imaging (interactive 3D scans)
- Virtual Reality (VR) and Augmented Reality (AR)

Memory Trick: Interactive = “**I Talk, Computer Answers back Instantly**” — think of it as a **conversation** (two-way), unlike Passive Graphics (a one-way lecture).

1.4 Passive Graphics (Non-Interactive Graphics)

Definition

Passive Computer Graphics (also called **Non-Interactive Graphics**) is a type of graphics system where the user has **no control** over the image once it has been generated. The image is created and displayed, but the user **cannot make any changes** to it in real time — it’s a “one-way” process.

Zero-Level Explanation: Think of watching a movie or a pre-recorded video. You can only watch it — you cannot pause it and redraw a character’s face, or make the car turn left instead of right. The graphic is **fixed** once created. This is Passive Graphics — display only, no interaction.

Key Characteristics of Passive Graphics

1. **One-way communication** — computer generates and displays, but user cannot alter it.
2. **No real-time control** by the user.
3. Once generated, the image/graphic **remains static/fixed** (in the sense that the user can't edit it interactively).
4. Simpler and requires **less processing power** than interactive graphics.

Examples of Passive Graphics

- Printed charts, graphs, or reports generated by software
- Pre-rendered movies/animation videos (like an animated film you just watch)
- Business graphs/pie-charts printed on paper
- Simple output-only displays (e.g., digital hoardings showing a fixed advertisement)

Advantages of Passive Graphics

- **Simple** to generate and requires **less computational power/hardware**.
- **Cheaper** to develop compared to interactive systems.
- Good for one-time display purposes such as **reports and presentations**.

Disadvantages of Passive Graphics

- **No user control** — cannot make changes or corrections without regenerating the whole image.
- **Less engaging** for the user compared to interactive graphics.
- Not suitable for applications requiring **real-time decision-making** (like games or simulations).

Applications of Passive Graphics

- Printed business graphs and reports
- Pre-recorded educational/promotional videos
- Static data visualization/analysis charts

Interactive vs Passive Graphics — Comparison Table (VERY IMPORTANT FOR EXAM)

Basis	Interactive Graphics	Passive Graphics
Communication	Two-way (user ↔ computer)	One-way (computer → user only)
User Control	User can modify/control the image in real time	User has no control once image is generated
Response Time	Real-time / immediate	No response needed
Hardware Requirement	High (needs fast CPU/GPU)	Low
Cost	Expensive	Cheaper
Examples	Video games, CAD, Flight simulators	Printed graphs, pre-recorded videos
Also called	—	Non-Interactive Graphics

Memory Trick: “Passive = Print & Forget” (like a printed photo — fixed, no editing) vs **“Interactive = Play & Control”** (like a video game — always responding to you).

Exam Tip: “Differentiate between Interactive and Passive Graphics” is **one of the most repeated questions** in BRABU exams (2/5 marks). Always present it as a table — examiners give quick marks for clear tabular comparison.

1.5 Applications of Computer Graphics

Computer Graphics is used in almost every field today. BRABU exams frequently ask **“Explain applications of Computer Graphics”** as a 5/10 mark question — this is one of the **most important topics** in Unit I.

1. Computer-Aided Design (CAD)

- Used by engineers and architects to design buildings, machine parts, electronic circuits, automobiles, etc.
- Software: AutoCAD, SolidWorks, CATIA.
- Allows designers to view objects from multiple angles, test different designs, and detect errors before actual construction.

2. Presentation Graphics

- Used to create summarized visual reports — bar charts, pie charts, line graphs — for business meetings, education, and research.
- Software: Microsoft PowerPoint, Excel Charts.
- Helps in **easy understanding of numerical/statistical data**.

3. Computer Art (Digital Art)

- Artists use computer graphics tools to create paintings, digital artwork, cartoons, and logos.
- Software: Adobe Photoshop, Illustrator, CorelDRAW.
- Includes both **fine art** (paintings) and **commercial art** (advertisements, logos, packaging design).

4. Entertainment (Movies, Animation, Gaming)

- Used to create **special effects (VFX)** in movies, animated films, and cartoons.
- Used to develop **2D and 3D video games**.
- Software: Blender, Maya, 3ds Max, Unreal Engine, Unity.
- Examples: Animated movies like Toy Story, special effects in Marvel movies.

5. Education and Training

- Used to create **interactive learning materials**, simulations, and training modules.
- **Flight simulators** train pilots without the risk of an actual flight.
- **Medical training simulators** allow doctors to practice surgery virtually.

6. Visualization (Scientific & Data Visualization)

- Used by scientists and researchers to visualize complex data — e.g., weather patterns, molecular structures, medical scans (MRI, CT scans).
- Helps in understanding **large and complex datasets** through graphs, charts, and 3D models.

7. Image Processing

- Involves modifying or enhancing existing images (different from Computer Graphics which *creates* images from data).
- Applications: photo editing, satellite image analysis, medical image enhancement (X-rays, MRI).

8. Graphical User Interface (GUI)

- Almost every modern software and operating system (Windows, macOS, Android) uses graphics to create icons, buttons, menus, and windows for easy user interaction.

9. Computer-Aided Manufacturing (CAM)

- Graphics used to control and monitor manufacturing machines and robotics in factories.

10. Business and Data Presentation

- Bar graphs, pie charts, histograms used to represent sales, profit/loss, growth data for quick business decision-making.

Table: Applications of Computer Graphics (Quick Revision)

Application Area	Example Software/Use
CAD	AutoCAD, SolidWorks (engineering design)
Presentation Graphics	PowerPoint, Excel Charts
Computer Art	Photoshop, CorelDRAW
Entertainment	Movies (VFX), Games (Blender, Unity)
Education/Training	Flight simulators, e-learning modules
Visualization	Scientific data, medical scans
Image Processing	Photo editing, satellite imaging
GUI	Windows, Android interfaces
CAM	Robotics, factory automation
Business Graphics	Sales charts, statistical graphs

Memory Trick: “CPCEEVIGCB” is hard to remember, so instead use the phrase: “**Clever Programmers Create Everything, Even Virtual Images, Games, Cars & Buildings**” — CAD, Presentation, Computer Art, Entertainment, Education, Visualization, Image Processing, GUI, CAM, Business.

IMPORTANT POINTS TO REMEMBER (UNIT I QUICK RECAP)

- Computer Graphics = creating, manipulating, displaying images using a computer.
- A Computer Graphics System = Input Devices + Processing Unit (CPU/GPU) + Storage (Frame Buffer) + Output Devices + Software.
- **Interactive Graphics** = two-way, real-time, user has control (e.g., games).
- **Passive Graphics** = one-way, no user control, fixed output (e.g., printed graphs).
- Applications span CAD, entertainment, education, medicine, business, art, and more.
- **Pixel** is the smallest unit of a digital image; **Frame Buffer** stores pixel data before display.

CHAPTER-END QUESTIONS — UNIT I

20 Short Questions (1-2 Marks)

1. Define Computer Graphics.
2. What is a pixel?
3. What is a Frame Buffer?
4. Define Computer Graphics System.
5. Name any four input devices used in graphics.
6. Name any three output devices used in graphics.
7. What is a GPU?
8. Define Interactive Graphics.
9. Define Passive Graphics.
10. Give two examples of Interactive Graphics.
11. Give two examples of Passive Graphics.
12. What is a Digitizer?
13. What is a Light Pen?
14. What is CAD? Expand the term.
15. What is CAM? Expand the term.
16. Name any two software used for Computer Art.
17. What is GUI?
18. What is resolution in computer graphics?
19. Name two applications of Computer Graphics in the entertainment industry.
20. What is the difference between Computer Graphics and Image Processing?

15 Long Questions (5-10 Marks)

1. Define Computer Graphics. Explain its importance and scope.
2. Explain the components of a Computer Graphics System with a neat diagram.
3. What is Interactive Graphics? Explain its working, advantages, and disadvantages.
4. What is Passive Graphics? Explain with examples, advantages, and disadvantages.
5. Differentiate between Interactive and Passive Graphics (tabular format).
6. Explain the applications of Computer Graphics in detail (at least 8 areas).
7. Explain the role of input and output devices in a Computer Graphics System.

8. Explain the use of Computer Graphics in CAD and CAM with examples.
9. Explain the use of Computer Graphics in entertainment and education.
10. What is a Frame Buffer? Explain its role in a graphics system.
11. Explain the difference between hardware and software components of a graphics system.
12. Discuss the use of Computer Graphics in scientific visualization and medical imaging.
13. Explain the concept of GUI and its relation to Computer Graphics.
14. What is the significance of GPU in modern computer graphics systems?
15. Explain how Computer Graphics is applied in business and presentation graphics.

10 Viva Questions

1. What is the full form of GUI?
2. What is the difference between CPU and GPU?
3. Give one real-life example of Interactive Graphics you use daily.
4. What does “real-time” mean in graphics?
5. Which device is used to convert a paper drawing to digital form?
6. Name one software used in movie special effects.
7. What is the smallest unit of a digital screen called?
8. What is a Plotter used for?
9. Why does Interactive Graphics need more processing power than Passive Graphics?
10. Give an example of Passive Graphics from daily life.

MCQs with Answers

1. Which of the following is an example of Interactive Graphics?
(a) Printed pie chart (b) Video game (c) Static poster (d) Pre-recorded video
Answer: (b) Video game
2. Frame Buffer is a part of:
(a) Input device (b) Output device (c) Memory/Storage (d) Software **Answer: (c) Memory/Storage**
3. CAD stands for:
(a) Computer Aided Design (b) Computer Application Design (c) Central Aided Design (d) Computer Art Design **Answer: (a) Computer Aided Design**
4. Which is NOT a characteristic of Passive Graphics?
(a) One-way communication (b) No user control (c) Real-time interaction (d) Static output **Answer: (c) Real-time interaction**
5. GPU stands for:
(a) General Processing Unit (b) Graphics Processing Unit (c) Graphic Program Utility (d) General Purpose Unit **Answer: (b) Graphics Processing Unit**
6. Which device is used for high-precision engineering line drawings?
(a) Printer (b) Plotter (c) Scanner (d) Monitor **Answer: (b) Plotter**
7. The smallest addressable point on a screen is called:
(a) Byte (b) Frame (c) Pixel (d) Vector **Answer: (c) Pixel**
8. Which of these is a Passive Graphics example?

- (a) CAD software (b) Flight simulator (c) Printed business report (d) Video game

Answer: (c) Printed business report

9. Which is an example of a graphics API/library?

- (a) MS Word (b) OpenGL (c) Notepad (d) Windows Explorer **Answer: (b) OpenGL**

10. Light Pen is used as a/an:

- (a) Output device (b) Input device (c) Storage device (d) Processing device **Answer: (b) Input device**

Previous-Year Style Questions (BRABU Pattern)

- Define Computer Graphics and explain its Computer Graphics System components. (10 marks — very frequently asked)
- Differentiate between Interactive Graphics and Passive Graphics. (5 marks — appears almost every year)
- Explain any five applications of Computer Graphics. (5/10 marks — very common)
- What is Interactive Graphics? Give examples. (2/5 marks)
- Write short notes on: (a) Frame Buffer (b) GPU (c) CAD (2 marks each — common short-note style question)

UNIT I — FINAL CHECKLIST (SYLLABUS COMPLIANCE)

Syllabus Topic	Status
1.1 Introduction	✓ Covered
Computer Graphics System	✓ Covered
Interactive Graphics	✓ Covered
Passive Graphics	✓ Covered
Applications of Computer Graphics	✓ Covered

✓ **100% of UNIT I syllabus is covered. No topic missing.**

End of Unit I. Please confirm to proceed to UNIT II (Display Devices, CRT, Bit-Mapped Graphics, Raster/Random Scan Displays, Color CRT Monitors, DVST, Plasma Panel, LED, LCD, Hard Copy Devices). # UNIT II — DISPLAY DEVICES

2.1 Introduction

Display devices are output devices that convert electrical signals from the computer into a visual image that we can see. This unit covers all major display technologies — from old CRT monitors to modern LED/LCD screens — along with how graphics are drawn and refreshed on them.

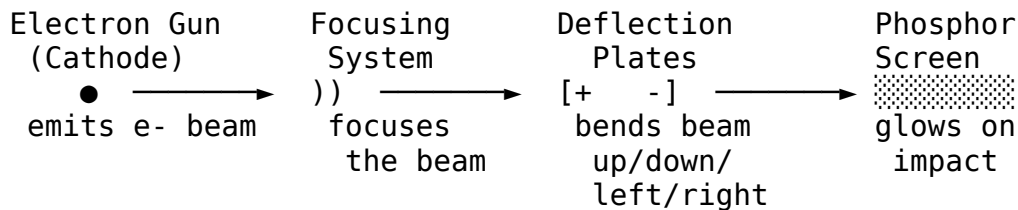
2.2 Cathode Ray Tube (CRT)

Definition

A **Cathode Ray Tube (CRT)** is a vacuum tube technology used in older television and computer monitors that produces images by firing a beam of electrons onto a phosphor-coated screen, causing it to glow.

Working Principle / Construction

A CRT consists of: 1. **Electron Gun** – emits (releases) a stream/beam of electrons using a heated cathode. 2. **Focusing System** – focuses the electron beam into a thin, sharp beam. 3. **Deflection System (Horizontal & Vertical plates/coils)** – bends/deflects the electron beam to hit the correct position on the screen. 4. **Phosphor-Coated Screen** – the inside of the screen is coated with phosphor material; when hit by the electron beam, it glows (emits light) for a very short time.



Working Steps:

1. The cathode (heated filament) releases a stream of electrons (electron gun).
2. The electron beam is focused into a fine beam by the focusing system.
3. The deflection system (using electric or magnetic fields) steers the beam to the desired (x, y) position on the screen.
4. When the beam strikes the phosphor-coated screen, the phosphor glows momentarily, creating a visible spot of light.
5. This process is repeated very fast (refreshing) to draw the complete picture and keep it visible (since phosphor glow fades quickly).

Persistence

Definition: Persistence is the time for which the phosphor glow remains visible on the screen after the electron beam is removed. Different phosphors have different persistence — high persistence phosphors are used for static images; low persistence for animation (needs fast refresh).

Advantages of CRT

- Good color accuracy and contrast.
- Relatively inexpensive to manufacture (historically).
- Fast response time (good for old-style gaming).

Disadvantages of CRT

- Bulky and heavy (takes a lot of desk space).
- High power consumption.
- Causes eye strain due to flickering (low refresh rates).
- Emits more radiation compared to modern displays.

Applications

- Old-generation computer monitors and televisions.
- Oscilloscopes (used in electronics labs even today for some uses).

Exam Tip: “Explain the working of CRT with a diagram” is a **very common 5/10 mark question**. Always draw the electron gun → focusing → deflection → phosphor screen diagram.

2.3 Bit-Mapped Graphics

Definition

Bit-Mapped Graphics (also called **Raster Graphics**) is a method of representing an image as a **matrix (grid) of individual pixels**, where each pixel's color/intensity is stored as a bit or a group of bits in memory (frame buffer).

Zero-Level Explanation: Imagine a graph paper with tiny squares. Each square (pixel) can be colored in. A bitmap image is exactly this — a grid where every little square has a specific color value stored in the computer's memory. A photograph on your phone is a bitmap/raster image (e.g., .jpg, .png, .bmp).

How it works:

- The screen is treated as a 2D grid of pixels.
- Each pixel has a corresponding memory location in the **frame buffer** that stores its color/intensity value.
- 1 bit per pixel = only 2 colors possible (black/white). More bits per pixel = more colors (this is called **color depth** or **bits per pixel, bpp**).

Bits per Pixel	Number of Colors Possible
1 bit	2 (black/white)
4 bits	16 colors
8 bits	256 colors
24 bits	16.7 million colors (True Color)

Advantages

- Can represent complex, photorealistic images (photographs) very well.
- Easy to edit pixel-by-pixel (used in photo editing).

Disadvantages

- **Loses quality when scaled up (zoomed)** — image becomes blurry/pixelated because you're just stretching the same fixed pixel grid.
- Takes more memory/storage for high resolution images compared to vector graphics.

Applications

- Digital photographs, scanned images, web images (JPEG, PNG, BMP formats).

Graphics Attributes

Definition: Graphics Attributes are properties that control **how** a graphical primitive (like a line, point, or character) is displayed on the screen — such as color, style, thickness, and pattern.

Common Graphics Attributes:

1. **Line Attributes:**
 - **Line Type** – solid, dashed, dotted.
 - **Line Width (Thickness)** – thin, medium, thick.
 - **Line Color** – the color of the drawn line.
2. **Color and Intensity Attributes:** Defines the color and brightness of a pixel/primitive.
3. **Area-Fill Attributes:** Defines the pattern (solid, hatched, patterned) used to fill a closed area/polygon.
4. **Character Attributes (Text Attributes):**
 - **Font** – style of the text (Arial, Times New Roman, etc.)
 - **Size** – size of the characters.
 - **Color** – color of the text.
 - **Orientation** – rotation angle of the text.

Exam Tip: “What are Graphics Attributes? Explain line and character attributes.” — commonly asked 5-mark question.

Refresh Cathode Ray Tubes (Refresh CRT)

Definition

A **Refresh CRT** is a type of CRT display where the image must be **redrawn (refreshed) repeatedly**, many times per second, to keep it visible on the screen — because the phosphor glow fades out quickly after being struck by the electron beam.

Key Concepts:

- **Refresh Rate:** The number of times per second the entire screen image is redrawn. Measured in Hertz (Hz). A typical refresh rate is 60 Hz (60 times per second) or higher for smooth, flicker-free viewing.
- **Refresh Buffer (Frame Buffer):** Stores the complete image data; the display controller reads from this buffer repeatedly to redraw the screen at the refresh rate.
- **Flicker:** If the refresh rate is too low, the human eye perceives a flickering image because the phosphor glow fades before the next refresh — this is uncomfortable to view.

Memory Trick: Refresh CRT = “**Draw it again and again before it fades away.**”

2.4 Random Scan Displays (Vector Scan / Stroke Writing Displays)

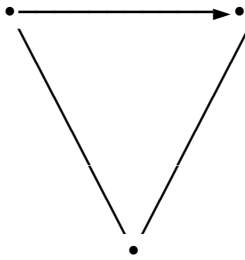
Definition

Random Scan Display is a display technology where the electron beam is directed **only to the parts of the screen where a picture is to be drawn**, tracing out the line segments of the image directly — like drawing with a pen, moving in any direction (not in a fixed pattern).

Working:

- The CRT beam is steered directly along the path of the line/graphic — jumping from one endpoint to another as instructed by the **display file (display list)**, which contains the graphics commands/coordinates.
- Since it draws only the lines that make up the picture (not the whole screen), it is also called **Vector Display** or **Line Drawing Display**.

Random Scan (vector) drawing a triangle:



(beam moves directly point-to-point, only drawing the outline)

Advantages

- Produces very **smooth lines** (high-quality line drawing) — no jagged/stair-step edges.
- Efficient for **line drawings** (does not waste time scanning empty/blank areas).

Disadvantages

- **Cannot display realistic, shaded, or filled (solid area) images** effectively — good for wireframe/line drawings only.
- Complex/detailed pictures cause flicker because the beam has too many lines to redraw within the refresh time.
- Expensive display processing hardware needed.

Applications

- Used in older CAD systems, engineering line-drawing applications.

Raster Scan Displays

Definition

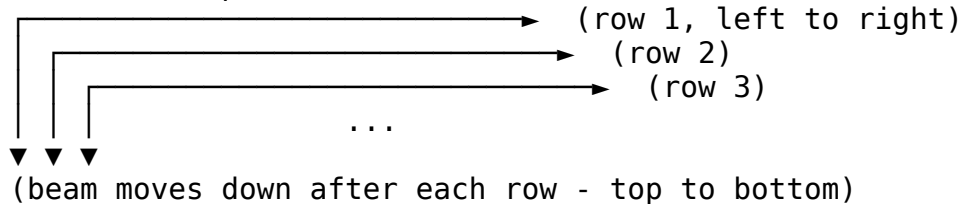
A **Raster Scan Display** is a display technology where the electron beam sweeps **across the entire screen row by row (left to right, top to bottom)**, like reading a book, illuminating the necessary pixels to form the image — this is the technology used in **almost all modern displays** (TVs, monitors).

Working:

- The screen is divided into a grid of pixels arranged in rows and columns.
- The electron beam scans each row from left to right, and moves down to the next row, continuing until the entire screen is covered — this complete sweep is called a **frame**.

- The **frame buffer** stores the intensity/color value for every pixel; as the beam scans each pixel position, it reads the corresponding value from the frame buffer and sets the brightness/color accordingly.
- After completing the bottom-right corner, the beam does a **Horizontal Retrace** (returns to the start of the next line) and a **Vertical Retrace** (returns from bottom-right to top-left to start the next frame).

Raster Scan pattern:



Interlaced vs Non-Interlaced Raster Scan

- **Non-Interlaced (Progressive) Scan:** All rows/lines of the screen are drawn in order, one after another, in a single pass.
- **Interlaced Scan:** The screen is refreshed in two passes — first all the **odd-numbered lines**, then all the **even-numbered lines**. This reduces flicker without needing to increase the refresh rate too much (used in older TVs).

Advantages

- Can display **realistic images** with shading, color gradients, and solid filled areas (photographs, videos).
- Relatively **inexpensive** electron-beam positioning hardware (fixed scanning pattern).

Disadvantages

- **Jagged/stair-step edges** on diagonal or curved lines — this defect is called **Aliasing** (fixed with Antialiasing techniques, covered in Unit IV).
- Requires a large **frame buffer memory** to store every pixel value.

Applications

- Almost all modern computer monitors, televisions, smartphone screens.

Raster Scan vs Random Scan — Comparison Table (VERY IMPORTANT)

Basis	Raster Scan	Random Scan (Vector)
Scanning Method	Row by row (entire screen)	Only along the lines of the picture
Resolution	Limited by pixel grid	Very high (smooth lines)
Realistic Images	Yes (supports shading, fill)	No (only outlines/wireframes)
Cost	Cheaper	Expensive
Frame Buffer Needed	Yes (stores entire screen)	Uses Display File (list of commands) instead
Common Use	Modern TVs/monitors	Old CAD/engineering

Basis	Raster Scan	Random Scan (Vector) systems
-------	-------------	------------------------------

Memory Trick: “Raster = Row-by-row (like reading a book).” “Random = drawn Randomly, only the picture lines (like tracing with a pen).”

Exam Tip: “Differentiate Raster Scan and Random Scan Displays” is **almost guaranteed to appear every year** at BRABU (5 marks) — memorize this table thoroughly.

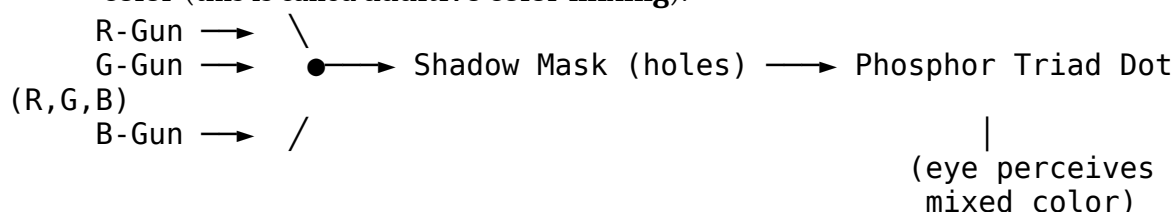
Color CRT Monitors

Definition

A **Color CRT Monitor** is a CRT display capable of producing **multiple colors** (not just black & white/monochrome) by using **three electron guns** and a specially coated phosphor screen with three types of color phosphor dots.

Working Principle — Shadow Mask Method (most common)

1. There are **three electron guns**, one each for **Red, Green, and Blue (RGB)** — the primary colors of light.
2. The screen phosphor coating is made up of tiny **triads (groups of 3 dots)** — one red-emitting, one green-emitting, one blue-emitting phosphor dot, very close together.
3. A **Shadow Mask** — a thin metal plate with tiny holes — is placed just behind the screen. It ensures that each electron gun’s beam can only strike its corresponding color phosphor dot (e.g., the “red” gun’s beam can only hit red phosphor dots).
4. By controlling the **intensity** of each of the three electron beams, different combinations of Red, Green, Blue light are produced, which our eyes blend together to perceive **any color** (this is called **additive color mixing**).



Advantages

- Can produce **millions of colors** through RGB mixing.
- Good color accuracy and image quality.

Disadvantages

- More **complex and expensive** than monochrome CRT.
- Shadow mask reduces brightness (some electrons are blocked by the mask).
- Still bulky (CRT technology limitation).

Applications

- Older color television sets and computer monitors.

2.5 Direct View Storage Tube (DVST)

Definition

A **Direct View Storage Tube (DVST)** is a type of CRT display that has the special ability to **retain (store) the displayed picture on the screen itself** using a special storage mesh, **without needing to be refreshed repeatedly** by the computer.

Working Principle:

1. Uses two electron guns: a **Primary Gun** (writes/draws the picture) and a **Flood Gun** (continuously floods the screen with low-energy electrons to keep the picture visible).
2. A **storage mesh (grid)** just behind the phosphor screen retains the pattern drawn by the primary gun.
3. The flood gun's electrons pass through the mesh only in the areas where the picture was drawn, continuously "re-lighting" that part of the phosphor — hence the image stays visible **without needing the frame buffer/refresh process**.

Advantages

- No refreshing needed → No frame buffer required → saves memory.
- No flicker, since the image doesn't need to be redrawn repeatedly.
- Can display very **complex images** without flicker issues (since complexity doesn't affect refresh time — there's no refresh!).

Disadvantages

- **Cannot selectively erase** a part of the picture — to make any change, the **entire screen must be erased and redrawn** from scratch.
- **No color capability** (DVST is monochrome only).
- Slower to make interactive changes (bad for dynamic/interactive graphics).
- Poor contrast compared to refresh CRT.

Applications

- Used in older CAD/engineering applications where the drawing is largely static (not frequently changing).

Exam Tip: DVST vs Refresh CRT comparison is a common question — DVST needs no refresh but cannot selectively erase; Refresh CRT needs constant refresh but allows dynamic/interactive editing.

Plasma Panel Displays (Gas-Discharge Displays)

Definition

A **Plasma Panel Display** is a flat-panel display technology that uses tiny cells filled with **ionized gas (plasma)** sandwiched between two glass plates; applying voltage to the gas causes it to emit light (glow), forming the image.

Working Principle:

1. The screen consists of a grid of small cells, each filled with a mixture of noble gases (like neon and xenon).
2. A grid of **electrode lines** (horizontal and vertical) runs between the glass plates.
3. When voltage is applied at the intersection of a horizontal and vertical electrode, the gas at that cell **ionizes and becomes plasma**, which emits **ultraviolet (UV) light**.
4. This UV light strikes a **phosphor coating** inside the cell, causing it to glow with visible light (similar to how a fluorescent tube works).

Advantages

- **Flat, thin design** (much slimmer than CRT).
- Good brightness, contrast, and wide viewing angles.
- No geometric distortion (unlike curved CRT screens).

Disadvantages

- **Higher power consumption** than LCD.
- Heavier than LCD/LED for the same screen size.
- Risk of “**screen burn-in**” (permanent ghost image) if a static image is displayed for a long time.
- Expensive to manufacture in small sizes.

Applications

- Large flat-screen televisions and display panels (though largely replaced by LED/LCD in modern times).

Thin Film Electroluminescent (TFEL) Displays

Definition

A **Thin Film Electroluminescent Display** is a flat-panel display technology where a thin film of **electroluminescent material** (such as zinc sulfide doped with manganese) is placed between two perpendicular sets of electrodes; applying voltage at the intersection causes the material at that point to glow.

Working Principle:

- Similar in structure to plasma panels: a grid of horizontal and vertical conductors, with the electroluminescent material sandwiched between them.
- When sufficient voltage is applied at an intersection, the material's electrons are excited and release energy in the form of light directly (electroluminescence) — **no gas ionization is needed** (this is the key difference from plasma panels).

Advantages

- Better resolution and image quality compared to plasma panels.
- Fast response time.

Disadvantages

- Requires **more power** than plasma panels to produce the same brightness.
- Manufacturing is complex and costly.

Applications

- Used in specialized industrial and military display equipment.

Memory Trick: Plasma = **gas glows (ionization)**. TFEL = **solid material glows directly (electroluminescence)** — no gas involved.

2.6 Light Emitting Diode (LED) Displays

Definition

An **LED Display** uses an array of **Light Emitting Diodes** — semiconductor devices that emit light when an electric current passes through them — as the picture elements (pixels) of the screen.

Working Principle:

- Each pixel (or a group of pixels) is made up of tiny LEDs — commonly a combination of Red, Green, and Blue LEDs (RGB LED) to produce full color.
- When current flows through the semiconductor material of the LED, electrons recombine with holes, releasing energy in the form of light (a phenomenon called **electroluminescence**).
- By varying the current/voltage, the brightness of each LED (and hence each pixel) can be controlled.

Note: “LED TV” in the market actually refers to an **LCD TV that uses LED backlighting** (not a true LED display, which is used mainly in large outdoor screens/billboards). True/direct LED displays are made entirely of LED pixels.

Advantages

- **Low power consumption**, energy efficient.
- Long lifespan.
- High brightness — good for outdoor displays even in sunlight.
- Compact and lightweight.

Disadvantages

- True LED displays (all-LED pixels) can be **expensive** for high-resolution small screens.
- Larger pixel pitch (spacing) in cheaper LED displays can reduce image sharpness at close viewing distance.

Applications

- Outdoor billboards, stadium display screens, digital signage.
- Backlighting for LCD TVs and monitors (“LED TVs”).

Liquid Crystal Display (LCD)

Definition

A **Liquid Crystal Display (LCD)** is a flat-panel display technology that uses **liquid crystal molecules**, which change their orientation when an electric voltage is applied, to control the passage of light through the screen and form images.

Working Principle:

1. LCD screens do not emit their own light (they are **non-emissive**) — they need a **backlight** (usually LED-based in modern screens) behind the panel.
2. The liquid crystal layer is sandwiched between two polarizing filters and glass plates with electrodes.
3. In the “off” (no voltage) state, the liquid crystal molecules twist light in a way that lets it pass through both polarizing filters.
4. When voltage is applied, the liquid crystal molecules **realign**, blocking the twisting of light, so light **cannot pass through** the second polarizer — this makes the pixel appear dark.
5. By controlling the voltage at each pixel (using a grid of transistors — Thin Film Transistor, TFT), and combining Red/Green/Blue color filters, a full-color image is formed.

Backlight → Polarizer 1 → Liquid Crystal Layer → Polarizer 2 → Color Filter → Screen

(voltage controls
light passage)

Advantages

- **Thin, lightweight, and compact** compared to CRT.
- **Low power consumption.**
- No geometric distortion, no flicker.
- Better for eye comfort over long usage.

Disadvantages

- Limited **viewing angles** (older LCDs) — image quality/color changes when viewed from the side.
- Needs a **separate backlight** source (since it doesn't emit its own light).
- Slower response time compared to LED/OLED for very fast-motion video (though modern LCDs have largely fixed this).

Applications

- Laptop screens, computer monitors, smartphone displays, digital watches, calculators.

CRT vs LCD vs LED vs Plasma — Comparison Table

Feature	CRT	LCD	LED	Plasma
Technology	Electron beam + phosphor	Liquid crystal + backlight	Light-emitting diodes	Ionized gas cells

Feature	CRT	LCD	LED	Plasma
Thickness	Bulky	Thin	Thin	Thin (but heavier than LCD)
Power Consumption	High	Low	Very Low	High
Image Quality	Good	Very Good	Excellent	Very Good (risk of burn-in)
Cost	Cheap (older tech)	Moderate	Moderate-High	Expensive

Hard Copy Output Devices

Definition: Hard Copy Output Devices are devices that produce a **permanent, physical (paper) copy** of the graphical output — as opposed to a screen display, which is a temporary/“soft copy.”

Types:

1. Printers:

- **Dot Matrix Printer:** Uses a matrix of pins striking an ink ribbon to print characters/images — cheap, noisy, low quality, but can print multi-part carbon forms.
- **Inkjet Printer:** Sprays tiny droplets of liquid ink onto paper — good quality, affordable, commonly used at home.
- **Laser Printer:** Uses a laser beam to create an image on a drum, which attracts toner (powder ink) that is then fused onto paper using heat — fast, high-quality, commonly used in offices.

2. Plotters:

- Used for high-precision **line drawings** — e.g., architectural blueprints, engineering designs, maps.
- Types: **Flatbed Plotter** (paper is stationary, pen moves in x-y directions) and **Drum Plotter** (paper moves on a rotating drum while pen moves side-to-side).

Comparison Table: Printers

Printer Type	Technology	Quality	Speed	Cost
Dot Matrix	Pins strike ribbon	Low	Slow	Very Cheap
Inkjet	Liquid ink spray	Good	Medium	Cheap-Moderate
Laser	Laser + toner + heat	Excellent	Fast	Moderate-High

Exam Tip: “Explain types of hard copy output devices” — commonly asked 5-mark question; mention Printers (3 types) and Plotters (2 types) with one line each.

IMPORTANT POINTS TO REMEMBER (UNIT II QUICK RECAP)

- **CRT** uses electron gun + phosphor screen; needs continuous refreshing.
- **Bit-Mapped/Raster Graphics** = pixel grid stored in frame buffer; more bits per pixel = more colors.
- **Random Scan (Vector)** draws only the picture's lines; **Raster Scan** draws the whole screen row by row — Raster Scan is used in all modern displays.
- **Color CRT** uses 3 electron guns (RGB) + shadow mask + phosphor triads.
- **DVST** stores the image on-screen (no refresh needed) but cannot selectively erase and has no color.
- **Plasma Panel** = ionized gas glows; **TFEL** = solid material glows directly (electroluminescence).
- **LED** = semiconductor pixels emit light directly; energy efficient, bright.
- **LCD** = liquid crystal controls light passage from a backlight; thin, low power, needs backlight.
- **Hard Copy devices** = Printers (Dot Matrix/Inkjet/Laser) and Plotters (Flatbed/Drum).

CHAPTER-END QUESTIONS — UNIT II

20 Short Questions

1. Define CRT.
2. What is persistence in a CRT?
3. Define Bit-Mapped Graphics.
4. What are Graphics Attributes? Name any two.
5. What is Refresh Rate?
6. Define Random Scan Display.
7. Define Raster Scan Display.
8. What is a Shadow Mask?
9. Define DVST.
10. What is a Flood Gun in DVST?
11. Define Plasma Panel Display.
12. What is TFEL?
13. Define LED Display.
14. Define LCD.
15. What is a backlight?
16. Name two types of Hard Copy Output Devices.
17. What is a Plotter used for?
18. Differentiate Dot Matrix and Laser printer (one line).
19. What is Interlaced Scanning?
20. What causes flicker in a display?

15 Long Questions

1. Explain the working of CRT with a labeled diagram.
2. Explain Bit-Mapped Graphics and Graphics Attributes in detail.

3. Differentiate between Random Scan and Raster Scan displays.
4. Explain the working of a Color CRT Monitor using the Shadow Mask method.
5. Explain DVST — working, advantages, disadvantages.
6. Explain Plasma Panel Display with its working principle.
7. Explain TFEL displays and differentiate from Plasma Panels.
8. Explain the working of LED displays with advantages and applications.
9. Explain the working of LCD with a diagram.
10. Compare CRT, LCD, LED, and Plasma displays in a table.
11. Explain Hard Copy Output devices — printers and plotters.
12. What is Refresh CRT? Explain the concept of refresh rate and flicker.
13. Explain interlaced vs non-interlaced raster scanning.
14. Explain the advantages and disadvantages of Raster Scan Displays.
15. Explain the construction and working of a modern flat panel display (any one: LCD/Plasma/LED).

10 Viva Questions

1. What is the function of an electron gun in a CRT?
2. Why does a CRT need continuous refreshing?
3. What are the primary colors used in a Color CRT?
4. Why can't DVST selectively erase part of an image?
5. Which display technology is used in most modern smartphones?
6. What is the role of a backlight in LCD?
7. Which printer type is fastest for offices?
8. What is aliasing in raster displays?
9. What is the difference between LED TV and true LED display?
10. Name one application of a Plotter.

MCQs with Answers

1. CRT uses which coating on the screen to produce light?
(a) Silicon (b) Phosphor (c) Plastic (d) Copper — **Answer: (b) Phosphor**
2. Raster Scan displays draw the image:
(a) Only along picture lines (b) Row by row across whole screen (c) In random order (d) Using gas ionization — **Answer: (b) Row by row across whole screen**
3. Which display does NOT need refreshing?
(a) Refresh CRT (b) Raster Scan (c) DVST (d) Color CRT — **Answer: (c) DVST**
4. Color CRT uses how many electron guns?
(a) 1 (b) 2 (c) 3 (d) 4 — **Answer: (c) 3**
5. Plasma Panel Displays use:
(a) Liquid Crystals (b) Ionized Gas (c) LEDs (d) Electron guns — **Answer: (b) Ionized Gas**
6. LCD requires a separate:

- (a) Electron gun (b) Backlight (c) Shadow mask (d) Storage mesh — **Answer: (b) Backlight**
7. Which printer uses a laser beam and toner?
(a) Dot Matrix (b) Inkjet (c) Laser Printer (d) Plotter — **Answer: (c) Laser Printer**
8. Vector/Random Scan display is best suited for:
(a) Photographs (b) Line drawings (c) Video playback (d) Gaming — **Answer: (b) Line drawings**
9. TFEL stands for:
(a) Thin Film Electroluminescent (b) Thin Frame Electric Line (c) Total Film Electric Layer (d) Thin Film Electric Line — **Answer: (a) Thin Film Electroluminescent**
10. Flatbed and Drum are types of:
(a) Printers (b) Monitors (c) Plotters (d) Scanners — **Answer: (c) Plotters**

Previous-Year Style Questions (BRABU Pattern)

- Explain the construction and working of CRT. (10 marks — very frequently asked)
- Differentiate Raster Scan and Random Scan Displays. (5 marks — appears almost every year)
- Explain DVST with advantages and disadvantages. (5 marks)
- Write short notes on: (a) Plasma Panel (b) LCD (c) LED (2 marks each)
- Explain Color CRT Monitor working (Shadow Mask method). (5/10 marks)

UNIT III — SCAN CONVERSION ALGORITHMS

3.1 Introduction

What is Scan Conversion?

Definition: Scan Conversion is the process of representing a continuous graphics object (like a line, circle, or ellipse) as a **discrete set of pixels** on a raster display — i.e., converting a mathematical/geometric description into the actual pixels that must be turned on to display that shape.

Zero-Level Explanation: When you draw a line on paper, it's smooth and continuous. But a computer screen is made of a grid of pixels (like graph paper) — you cannot draw a “perfectly smooth” diagonal line, you can only light up certain squares. Scan Conversion algorithms decide **exactly which pixels** to light up so that the result *looks like* the line/circle/shape you wanted.

This unit is the **most numerical-heavy** unit — expect direct algorithm-based numerical questions in the exam.

3.2 Digital Differential Analyzer (DDA) Algorithm

Theory

The **DDA Algorithm** is a simple line-drawing algorithm that uses the **line's slope** to calculate successive pixel positions incrementally — it calculates one coordinate (x or y) and increments the other based on the slope.

Formula

Given two endpoints (x_1, y_1) and (x_2, y_2) :

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

$$\text{Steps} = \max(|dx|, |dy|) \quad [\text{number of pixels to plot}]$$

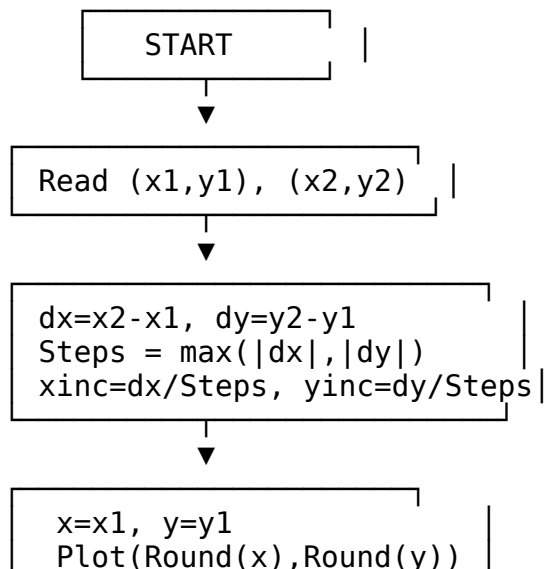
$$x_increment = dx / \text{Steps}$$

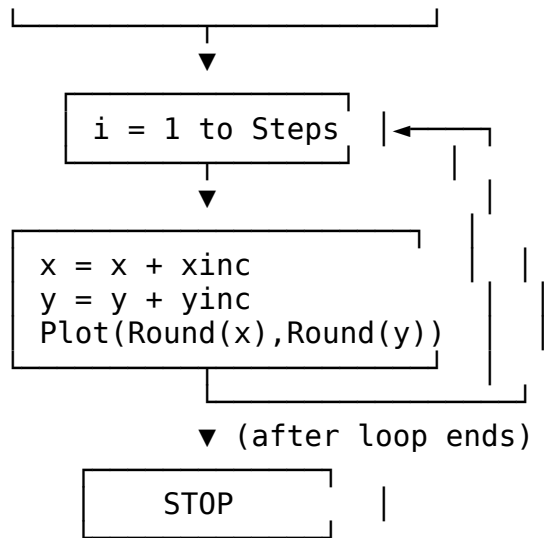
$$y_increment = dy / \text{Steps}$$

Step-by-Step Algorithm

1. Read the line endpoints (x_1, y_1) and (x_2, y_2) .
2. Calculate $dx = x_2 - x_1$ and $dy = y_2 - y_1$.
3. Calculate $\text{Steps} = \max(|dx|, |dy|)$.
4. Calculate $x_increment = dx / \text{Steps}$, $y_increment = dy / \text{Steps}$.
5. Set $x = x_1$, $y = y_1$.
6. Plot the pixel at $(\text{Round}(x), \text{Round}(y))$.
7. Repeat Steps times:
 - $x = x + x_increment$
 - $y = y + y_increment$
 - Plot the pixel at $(\text{Round}(x), \text{Round}(y))$
8. Stop.

Flowchart (ASCII)





Solved Numerical Example — DDA

Draw a line from (2, 3) to (8, 9).

$$dx = 8 - 2 = 6$$

$$dy = 9 - 3 = 6$$

$$\text{Steps} = \max(|6|, |6|) = 6$$

$$x_increment = 6/6 = 1$$

$$y_increment = 6/6 = 1$$

Step	x	y	Plotted Pixel (Round)
0	2	3	(2, 3)
1	3	4	(3, 4)
2	4	5	(4, 5)
3	5	6	(5, 6)
4	6	7	(6, 7)
5	7	8	(7, 8)
6	8	9	(8, 9)

Final Answer — Pixels plotted: (2,3), (3,4), (4,5), (5,6), (6,7), (7,8), (8,9)

Time Complexity

$O(n)$, where n = number of pixels (Steps) — linear time.

Advantages of DDA

- Simple and easy to understand/implement.
- Faster than direct use of the line equation ($y = mx + c$) for every point.

Disadvantages of DDA

- Uses **floating-point arithmetic** (real number addition and rounding) — this makes it **slower** and introduces **rounding errors**, leading to lines that aren't perfectly accurate (slight zig-zag on some slopes).
- Rounding off operation is time-consuming.

Applications

- Basic line drawing in simple graphics systems (though Bresenham's algorithm, below, is generally preferred in practice).

Exam Tip: DDA numericals are **extremely common** (5/10 marks) — always show the table format above (Step, x, y, Plotted Pixel). Memorize the formula for Steps = $\max(|dx|, |dy|)$.

3.3 Bresenham's Line Drawing Algorithm

Theory

Bresenham's Line Algorithm is an efficient line-drawing algorithm that uses **only integer arithmetic** (addition, subtraction, and multiplication by 2) to determine the closest pixels to the ideal line path — making it much faster than DDA (no floating-point division/rounding needed).

Key Idea: At each step, the algorithm decides whether to move to the next pixel **straight (east)** or **diagonally (north-east)** based on a **decision parameter** that tracks how far the true line is from the middle of the two candidate pixels.

Integer Bresenham Algorithm (for slope $0 < m < 1$, i.e., $|dx| > |dy|$)

Formula / Decision Parameter

$$dx = x_2 - x_1$$

$$dy = y_2 - y_1$$

Initial Decision Parameter: $p_0 = 2*dy - dx$

At each step k:

if $p_k < 0$:

Next point = (x_{k+1}, y_k) [move East]

$$p(k+1) = p_k + 2*dy$$

else:

Next point = (x_{k+1}, y_{k+1}) [move North-East]

$$p(k+1) = p_k + 2*dy - 2*dx$$

Step-by-Step Algorithm

1. Read the line endpoints (x_1, y_1) and (x_2, y_2) .
2. Calculate $dx = x_2 - x_1$, $dy = y_2 - y_1$.
3. Calculate initial decision parameter: $p_0 = 2*dy - dx$.
4. Plot the first point (x_1, y_1) .
5. At each step $k = 0, 1, 2, \dots$ until $x = x_2$:
If $p_k < 0$:

```

        Plot (xk+1, yk)
        p(k+1) = pk + 2dy
    Else:
        Plot (xk+1, yk+1)
        p(k+1) = pk + 2dy - 2dx
6. Repeat until the endpoint (x2, y2) is reached.
7. Stop.

```

Solved Numerical Example — Bresenham Line

Draw a line from (2, 2) to (8, 5) using Bresenham's Algorithm.

```

dx = 8 - 2 = 6
dy = 5 - 2 = 3
Initial p0 = 2*dy - dx = 2(3) - 6 = 0
2dy = 6
2dy - 2dx = 6 - 12 = -6

```

k	pk	Decision (pk<0?)	Next Point (x,y)	p(k+1)
0	p0=0	No (0 is not <0)	(3, 3)	0 + (-6) = -6
1	-6	Yes	(4, 3)	-6+6 = 0
2	0	No	(5, 4)	0+(-6) = -6
3	-6	Yes	(6, 4)	-6+6 = 0
4	0	No	(7, 5)	0+(-6) = -6
5	-6	Yes	(8, 5)	— (endpoint reached)

Final Answer — Pixels plotted: (2,2), (3,3), (4,3), (5,4), (6,4), (7,5), (8,5)

Time Complexity

$O(n)$ — linear, but faster in practice than DDA since it avoids floating-point operations (uses only integer add/subtract).

Advantages of Bresenham's Algorithm

- Uses **only integer arithmetic** — no floating-point division or rounding — very fast and efficient.
- More **accurate** than DDA (produces the mathematically closest pixel to the true line every time).
- Widely used in real graphics hardware/software.

Disadvantages

- Slightly more complex to understand/derive than DDA.
- The basic version only handles lines with slope between 0 and 1 ($|dx| > |dy|$, both positive) — needs modification (General Bresenham) for other slopes/directions.

Applications

- Used in almost all modern graphics libraries/hardware (OpenGL, printer drivers, etc.) for line drawing.

Exam Tip: Bresenham numerals are **the single most important topic** in this unit — practice at least 3-4 different examples by hand. Always show the decision parameter table.

General Bresenham Algorithm (All Slopes/Octants)

Theory

The basic Bresenham algorithm only works correctly for lines in the **first octant** (slope between 0 and 1, moving right and up). The **General Bresenham Algorithm** extends the method to work for lines with **any slope** (in all 8 octants — any direction: steep, shallow, negative slope, etc.) by considering the sign of dx and dy and swapping the roles of x and y when necessary (i.e., stepping along whichever axis has the greater change).

Key Modifications for General Case:

1. If $|dy| > |dx|$ (steep line, slope > 1): step along the y-axis instead of x-axis (swap x and y roles).
2. If **dx is negative**: swap the two endpoints and process it as swapped (or decrement instead of increment x).
3. If **dy is negative**: decrement y instead of incrementing (moving downward).

Table: Bresenham Adjustments by Octant

Condition	Action
$0 \leq \text{slope} \leq 1, x_2 > x_1$	Standard case (increment x, decide y)
slope > 1	Swap x/y roles (increment y, decide x)
slope negative	Decrement y (or x) instead of increment
$x_2 < x_1$	Swap endpoints first, then apply algorithm

Exam Tip: For BRABU-level exams, you are usually only asked to solve numerals in the **first octant** (like the example above) — but you should be able to **explain in words** how the algorithm generalizes to other octants (mention swapping x/y roles based on slope).

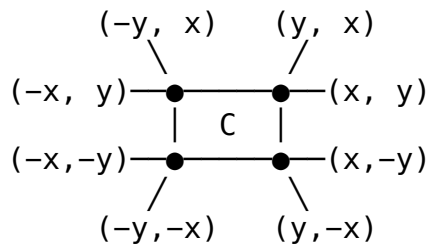
3.4 Circle Generation Algorithms

Why not use the direct circle equation?

The circle equation $x^2 + y^2 = r^2$ could theoretically be used directly to calculate y for every x, but this involves **expensive floating-point square root and multiplication** operations, and produces **unevenly spaced pixels** (denser near the top/bottom of the circle) — so efficient integer-based algorithms are used instead.

8-Way Symmetry Property of a Circle

A circle has perfect symmetry — if you calculate just **one point** (x, y) on the circle in the first octant (45° segment), you automatically know **7 other points** by symmetry:



Given a point (x, y) on the circle, the other 7 symmetric points are: (y, x) , $(y, -x)$, $(-y, x)$, $(-y, -x)$, $(x, -y)$, $(-x, y)$, $(-x, -y)$

This means an algorithm only needs to calculate points for **one octant (45°)**, saving $7/8$ of the computation.

Memory Trick: “A circle is like a pizza cut into 8 equal slices — calculate 1 slice, mirror it 7 times!”

Bresenham Circle Generation Algorithm

Theory

Similar in spirit to Bresenham’s Line Algorithm — uses **integer arithmetic and a decision parameter** to choose between two candidate pixels at each step while tracing one octant (45°) of the circle, then uses 8-way symmetry to plot the rest.

Formula

Decision Parameter (initial): $p_0 = 1 - r$ (using midpoint variant; classic Bresenham uses $p_0 = 3 - 2r$ in some textbooks)

At each step, starting from $(x_0, y_0) = (0, r)$:

```
if  $p_k < 0$ :
    Next point:  $(x_{k+1}, y_k)$  [move East]
     $p(k+1) = p_k + 2x_{k+1} + 1...$  (formulas vary slightly by textbook version)
else:
    Next point:  $(x_{k+1}, y_{k+1})$  [move South-East]
     $p(k+1) = p_k + 2x_{k+1} + 1 - 2y_{k+1}$ 
```

(Note: Most Indian university syllabi, including BRABU, primarily expect the **Midpoint Circle Algorithm** below for numericals — it is mathematically equivalent to Bresenham’s Circle Algorithm and is the standard expected answer.)

Midpoint Circle Algorithm

Theory

The **Midpoint Circle Algorithm** generates a circle by testing the **midpoint between two candidate pixels** against the circle's equation to decide which pixel is closer to the true circle path. It traces just one octant (0° to 45°) and uses 8-way symmetry for the rest.

Formula

Circle Function: $f(x,y) = x^2 + y^2 - r^2$ - If $f(x,y) < 0$ → point is **inside** the circle. - If $f(x,y) > 0$ → point is **outside** the circle. - If $f(x,y) = 0$ → point is **on** the circle.

Decision Parameter:

Initial: $p_0 = 1 - r$

At each step k , starting from $(x_0, y_0) = (0, r)$:

```
if  $p_k < 0$ :
    Next point =  $(x_{k+1}, y_k)$            [move East – choose pixel E]
     $p_{k+1} = p_k + 2x_{k+1} + 1$ 
else:
    Next point =  $(x_{k+1}, y_{k-1})$        [move South-East – choose
pixel SE]
     $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$ 
```

(where $x_{k+1} = x_k + 1$)

Stop when $x \geq y$ (i.e., the 45° boundary of the octant is reached).

Step-by-Step Algorithm

1. Read radius r ; set center (typically $(0,0)$, then translate).
2. Set $x=0$, $y=r$.
3. Calculate initial decision parameter $p_0 = 1 - r$.
4. Plot the first point $(0, r)$ and its 7 symmetric points.
5. At each step, while $x < y$:
 If $p_k < 0$:
 $x = x+1$
 $p_{k+1} = p_k + 2x + 1$
 Else:
 $x = x+1$; $y = y-1$
 $p_{k+1} = p_k + 2x + 1 - 2y$
 Plot (x,y) and its 7 symmetric points.
6. Stop when $x \geq y$.

Solved Numerical Example — Midpoint Circle

Draw a circle with radius $r = 10$, centered at origin.

Initial: $x_0=0$, $y_0=10$

$p_0 = 1 - r = 1 - 10 = -9$

k	p_k	Decision	(x_{k+1}, y_{k+1})	$p(k+1)$
0	-9	$p_k < 0 \rightarrow E$	(1, 10)	$-9 + 2(1) + 1 = -6$
1	-6	$p_k < 0 \rightarrow E$	(2, 10)	$-6 + 2(2) + 1 = -1$
2	-1	$p_k < 0 \rightarrow E$	(3, 10)	$-1 + 2(3) + 1 = 6$
3	6	$p_k \geq 0 \rightarrow SE$	(4, 9)	$6 + 2(4) + 1 - 2(9) = -3$
4	-3	$p_k < 0 \rightarrow E$	(5, 9)	$-3 + 2(5) + 1 = 8$
5	8	$p_k \geq 0 \rightarrow SE$	(6, 8)	$8 + 2(6) + 1 - 2(8) = 5$
6	5	$p_k \geq 0 \rightarrow SE$	(7, 7)	stop, $x=y$ reached

Final Answer — Points in first octant: (0,10),(1,10),(2,10),(3,10),(4,9),(5,9),(6,8),(7,7)

Each of these points is mirrored using 8-way symmetry to complete the full circle.

Time Complexity

$O(r)$ — proportional to the radius (linear).

Advantages

- Uses **only integer arithmetic** — fast and efficient.
- Exploits 8-way symmetry — only 1/8th of the circle needs to be calculated.

Disadvantages

- Only works for circles (not ellipses directly — needs a modified algorithm).
- Slightly complex decision-parameter derivation for students to memorize.

Applications

- Drawing circles/arcs in graphics software, CAD tools, GUI rendering.

Exam Tip: Midpoint Circle Algorithm numerical is **one of the most important/guaranteed questions** in BRABU exams (10 marks) — practice the table format thoroughly, and always mention the “stop when $x \geq y$ ” condition and the 8-way symmetry plotting.

3.5 Ellipse Generation Algorithm — Midpoint Ellipse Algorithm

Theory

An ellipse is like a “stretched circle” with two different radii: **r_x** (along the x-axis) and **r_y** (along the y-axis). The **Midpoint Ellipse Algorithm** works similarly to the Midpoint Circle Algorithm but must handle **two regions separately** because the slope of the ellipse curve changes differently than a circle’s.

Ellipse Equation

Ellipse function: $f(x,y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2$
 - $f(x,y) < 0 \rightarrow$ inside the ellipse

- $f(x,y) > 0 \rightarrow$ outside the ellipse
- $f(x,y) = 0 \rightarrow$ on the ellipse

4-Way Symmetry of an Ellipse

Unlike a circle (8-way symmetry), an ellipse only has **4-way symmetry** (since $r_x \neq r_y$ generally): Given point (x, y) on the ellipse, the other 3 symmetric points are: $(-x, y)$, $(x, -y)$, $(-x, -y)$

Two Regions of the Ellipse

Because the slope (dy/dx) of the ellipse curve is -1 at the boundary between the regions, the algorithm is split into: - **Region 1:** Where the ellipse's slope is greater than -1 (near the top, close to x-axis progression) — step in **x direction**. - **Region 2:** Where the slope is less than -1 (near the side, close to y-axis progression) — step in **y direction**.

Formula (Region 1)

Initial Decision Parameter: $p1_0 = r_y^2 - r_x^2 r_y + (r_x^2/4)$

At each step (while decision boundary not reached):

```

if  $p1_k < 0$ :
    Next point =  $(x_{k+1}, y_k)$ 
     $p1(k+1) = p1_k + 2r_y^2 x_{k+1} + r_y^2$ 
else:
    Next point =  $(x_{k+1}, y_{k-1})$ 
     $p1(k+1) = p1_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$ 

```

Formula (Region 2)

Initial Decision Parameter: $p2_0 = r_y^2(x+0.5)^2 + r_x^2(y-1)^2 - r_x^2 r_y^2$

At each step:

```

if  $p2_k > 0$ :
    Next point =  $(x_k, y_{k-1})$ 
     $p2(k+1) = p2_k - 2r_x^2 y_{k+1} + r_x^2$ 
else:
    Next point =  $(x_{k+1}, y_{k-1})$ 
     $p2(k+1) = p2_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$ 

```

Step-by-Step Algorithm (Summary)

1. Read r_x, r_y .
2. Region 1: Start at $(0, r_y)$. Calculate $p1_0$. Step in +x direction
while
 $2*r_y^2*x < 2*r_x^2*y$, choosing E or SE pixel based on $p1_k$. Plot with 4-way symmetry.
3. Region 2: Continue from where Region 1 stopped. Step in -y direction
until $y=0$, choosing S or SE pixel based on $p2_k$. Plot with 4-way symmetry.
4. Stop.

Solved Numerical Example — Midpoint Ellipse (Region 1 only, simplified)

Given $r_x = 8$, $r_y = 6$. Find the first few points in Region 1.

$$\begin{aligned} p1_0 &= r_y^2 - r_x^2 r_y + r_x^2 / 4 \\ &= 36 - (64)(6) + 64/4 \\ &= 36 - 384 + 16 = -332 \end{aligned}$$

k	x,y (start)	p1k	Decision	Next Point
0	(0,6)	-332	$p1k < 0 \rightarrow E$	(1,6)
1	(1,6)	$p1_1 = -332 + 2(36)(1) + 36 = -224$	$p1k < 0 \rightarrow E$	(2,6)
2	(2,6)	$p1_2 = -224 + 2(36)(2) + 36 = -80$	$p1k < 0 \rightarrow E$	(3,6)

(Continue similarly until the region-switch condition $2r_y^2 x \geq 2r_x^2 y$ is met, then switch to Region 2 formulas.)

Time Complexity

$O(r_x + r_y)$ — linear with respect to the two radii.

Advantages

- Uses integer arithmetic (efficient, like Bresenham/Midpoint Circle).
- Handles the changing slope of an ellipse correctly using two regions.

Disadvantages

- More complex than the circle algorithm (two regions, two sets of formulas to remember).
- Requires care in switching between Region 1 and Region 2 at the correct point.

Applications

- Drawing ellipses/ovals in graphics software, CAD, and design tools.

Exam Tip: BRABU often asks to “Explain the Midpoint Ellipse Algorithm” (theory + formulas) for 10 marks; full numerical solving of both regions is lengthy, so many students are expected to demonstrate the **method and Region 1 calculation** at minimum — always mention the 4-way symmetry and the two-region concept clearly, as these are the key differentiators examiners look for.

Arc Generation Algorithm

Theory

An **Arc** is simply a **portion (part) of a circle or ellipse** between two specified angles, rather than the complete curve. Arc generation uses the same Midpoint Circle/Ellipse algorithms, but

plotting is restricted to only the required angular range (start angle to end angle), instead of using full 8-way/4-way symmetry for the entire shape.

Method:

1. Use the Midpoint Circle (or Ellipse) Algorithm to compute points octant-by-octant.
2. Convert the given start and end angles into the corresponding **octant boundaries and (x,y) coordinate limits**.
3. Only plot the points that fall **within the specified angular range** — skip the symmetry reflections that fall outside the desired arc.

Applications

- Drawing partial curves — e.g., pie-chart slices, rounded rectangle corners, dial gauges in UI design.

Fill Algorithms (Area Filling)

Definition

Fill Algorithms are used to color/fill the interior of a closed figure (polygon, circle, etc.) with a specified color or pattern.

1. Boundary Fill Algorithm

Theory: Starts from a **seed point** inside the region and spreads outward, coloring pixels, until it hits the **boundary color** — used when the boundary of the region is defined by a **single specific color**.

Algorithm (4-connected version):

```
BoundaryFill4(x, y, fill_color, boundary_color):
    if getPixelColor(x,y) != boundary_color AND getPixelColor(x,y) !=
fill_color:
        setPixelColor(x, y, fill_color)
        BoundaryFill4(x+1, y, fill_color, boundary_color)
        BoundaryFill4(x-1, y, fill_color, boundary_color)
        BoundaryFill4(x, y+1, fill_color, boundary_color)
        BoundaryFill4(x, y-1, fill_color, boundary_color)
```

- **4-connected:** Checks only 4 neighboring pixels (up, down, left, right).
- **8-connected:** Checks all 8 neighboring pixels (includes diagonals) — better for irregular boundaries, avoids “leaking” through diagonal gaps.

Disadvantage: Can fail (infinite loop or stack overflow) on very large areas due to deep recursion; can “leak” out through gaps in the boundary if not 8-connected for a diagonal-boundary shape.

2. Flood Fill Algorithm

Theory: Similar to Boundary Fill, but instead of stopping at a specific boundary *color*, it replaces **all connected pixels of a specific old/interior color** with the new fill color —

useful when the region's interior is **already a single uniform color** (rather than having a distinctly colored border).

Algorithm (4-connected version):

```
FloodFill4(x, y, new_color, old_color):  
    if getPixelColor(x,y) == old_color:  
        setPixelColor(x, y, new_color)  
        FloodFill4(x+1, y, new_color, old_color)  
        FloodFill4(x-1, y, new_color, old_color)  
        FloodFill4(x, y+1, new_color, old_color)  
        FloodFill4(x, y-1, new_color, old_color)
```

Boundary Fill vs Flood Fill — Comparison Table

Basis	Boundary Fill	Flood Fill
Stopping condition	Stops at a specific boundary color	Replaces all pixels of a specific old interior color
Use case	Region has a distinctly colored border	Region's interior is a uniform color (no distinct border needed)
Risk	May leak through gaps in border	May overflow into unintended connected regions of the same old color

3. Scan-Line Polygon Fill Algorithm

Theory: Fills a polygon by processing it **row by row (scan line by scan line)**. For each horizontal scan line, it finds the **intersection points** of the scan line with the polygon's edges, sorts them by x-coordinate, and fills the pixels **between pairs of intersection points**.

Steps:

1. Find the minimum and maximum y-coordinates of the polygon.
2. For each scan line y (from ymin to ymax):
 - a. Find all x-coordinates where this scan line intersects the polygon edges.
 - b. Sort these x-intersection points in increasing order.
 - c. Fill pixels between each pair of intersection points (1st-2nd, 3rd-4th, etc.)
3. Move to the next scan line, repeat.

Advantage: Efficient and works well for filling polygons of any shape (convex or concave) without recursion issues (unlike Boundary/Flood fill).

Applications: Used extensively in rendering filled polygons in graphics software, games, and CAD systems.

IMPORTANT POINTS TO REMEMBER (UNIT III QUICK RECAP)

- **DDA** uses floating-point increments based on slope; simple but slower/less accurate.

- **Bresenham's Line Algorithm** uses only integer arithmetic via a decision parameter; faster and more accurate — the industry standard.
- **General Bresenham** extends the basic algorithm to all octants/slopes by swapping x/y roles as needed.
- **Circle Generation** exploits **8-way symmetry** — only 1 octant needs to be calculated (Midpoint Circle Algorithm, decision parameter $p_0 = 1-r$).
- **Ellipse Generation** exploits **4-way symmetry** and uses **2 regions** (different formulas) due to the changing slope.
- **Arc Generation** = partial circle/ellipse, restricted by start/end angles.
- **Fill Algorithms:** Boundary Fill (stops at border color), Flood Fill (replaces old interior color), Scan-Line Fill (row-by-row, most efficient for polygons).

CHAPTER-END QUESTIONS — UNIT III

20 Short Questions

1. Define Scan Conversion.
2. What is the formula for Steps in DDA algorithm?
3. What are the two disadvantages of DDA?
4. Define the decision parameter in Bresenham's algorithm.
5. Why is Bresenham's algorithm faster than DDA?
6. What is 8-way symmetry in a circle?
7. What is the initial decision parameter in Midpoint Circle Algorithm?
8. What is 4-way symmetry in an ellipse?
9. Why does the Ellipse algorithm need two regions?
10. Define Arc.
11. What is Boundary Fill Algorithm?
12. What is Flood Fill Algorithm?
13. Differentiate 4-connected and 8-connected fill.
14. What is Scan-Line Polygon Fill?
15. Give the circle equation used in Midpoint Circle Algorithm.
16. What is the stopping condition in Midpoint Circle Algorithm?
17. What type of arithmetic does Bresenham's algorithm use?
18. What is a seed point?
19. Name any one application of the Arc Generation Algorithm.
20. What is the time complexity of the DDA algorithm?

15 Long Questions

1. Explain DDA algorithm with steps and a solved numerical example.
2. Explain Bresenham's Line Drawing Algorithm with a solved numerical example.
3. Compare DDA and Bresenham's Line Algorithm.
4. Explain the General Bresenham Algorithm for all octants.
5. Explain 8-way symmetry of a circle with a diagram.
6. Explain the Midpoint Circle Algorithm with a solved numerical example.

7. Explain the Midpoint Ellipse Algorithm with formulas for both regions.
8. Explain 4-way symmetry of an ellipse.
9. Explain Arc Generation Algorithm.
10. Explain Boundary Fill Algorithm with pseudocode.
11. Explain Flood Fill Algorithm with pseudocode.
12. Differentiate Boundary Fill and Flood Fill Algorithm.
13. Explain Scan-Line Polygon Fill Algorithm with steps.
14. Derive the decision parameter formula for Bresenham's Line Algorithm. (Theory-based)
15. Explain why circle/ellipse algorithms avoid using the direct equation (with square root).

10 Viva Questions

1. What does DDA stand for?
2. Which algorithm uses only integer arithmetic for line drawing?
3. What is the initial decision parameter formula in Bresenham's Line Algorithm?
4. How many octants does a circle have?
5. Why is 8-way symmetry useful?
6. What condition stops the Midpoint Circle Algorithm loop?
7. Name the two regions in the Midpoint Ellipse Algorithm.
8. What is the main difference between Boundary Fill and Flood Fill?
9. Which fill algorithm is best for large complex polygons?
10. What is the risk of 4-connected boundary fill?

MCQs with Answers

1. DDA stands for:
 - (a) Digital Differential Analyzer (b) Direct Data Algorithm (c) Digital Drawing Analyzer (d) Data Differential Algorithm — **Answer: (a) Digital Differential Analyzer**
2. Bresenham's Line Algorithm primarily uses:
 - (a) Floating-point arithmetic (b) Integer arithmetic (c) Trigonometric functions (d) Matrix multiplication — **Answer: (b) Integer arithmetic**
3. A circle has how many symmetric octants?
 - (a) 4 (b) 6 (c) 8 (d) 12 — **Answer: (c) 8**
4. The initial decision parameter in Midpoint Circle Algorithm is:
 - (a) $p_0 = r-1$ (b) $p_0 = 1-r$ (c) $p_0 = 2r-1$ (d) $p_0 = r/2$ — **Answer: (b) $p_0 = 1-r$**
5. An ellipse has how many symmetric quadrants?
 - (a) 2 (b) 4 (c) 6 (d) 8 — **Answer: (b) 4**
6. Flood Fill Algorithm replaces:
 - (a) Only boundary pixels (b) All pixels of a specific old color (c) Only corner pixels (d) Random pixels — **Answer: (b) All pixels of a specific old color**
7. Scan-Line Fill Algorithm processes the polygon:
 - (a) Pixel by pixel randomly (b) Row by row (c) Column by column (d) Diagonally — **Answer: (b) Row by row**

8. In DDA, Steps =
 (a) $\min(|dx|, |dy|)$ (b) $dx+dy$ (c) $\max(|dx|, |dy|)$ (d) $dx*dy$ — **Answer: (c) $\max(|dx|, |dy|)$**
9. Boundary Fill Algorithm stops when it reaches:
 (a) A specific boundary color (b) The center of the shape (c) A random pixel (d) The seed point again — **Answer: (a) A specific boundary color**
10. Midpoint Ellipse Algorithm requires how many regions?
 (a) 1 (b) 2 (c) 3 (d) 4 — **Answer: (b) 2**

Previous-Year Style Questions (BRABU Pattern)

- Explain DDA Algorithm with a suitable example. (10 marks — very frequently asked)
- Explain Bresenham's Line Drawing Algorithm with numerical. (10 marks — guaranteed almost every year)
- Explain Midpoint Circle Generation Algorithm with example. (10 marks — very frequently asked)
- Differentiate Boundary Fill and Flood Fill Algorithm. (5 marks)
- Write short notes on Midpoint Ellipse Algorithm. (5 marks)

UNIT IV — ANTIALIASING, 2D TRANSFORMATIONS & CLIPPING

4.1 Fundamentals of Antialiasing

What is Aliasing?

Definition: Aliasing is the **jagged, stair-step (zigzag) appearance** that occurs when a smooth diagonal or curved line is drawn on a raster (pixel-grid) display, because pixels are discrete square units and cannot perfectly represent a continuous smooth curve/line.

Ideal Line:



Aliased (jagged) Line on raster grid:



What is Antialiasing?

Definition: Antialiasing is a technique used to **reduce or eliminate the jagged/stair-step effect** (aliasing) by adjusting pixel intensities/colors along the edges of lines and shapes, making them appear smoother to the human eye.

How Antialiasing Works (Basic Concept):

- Instead of a pixel being purely “ON” (full color) or “OFF” (background), antialiasing calculates the **percentage of the pixel area covered** by the line/shape and sets the pixel's intensity **proportionally** (partial/blended color) — this tricks the human eye into perceiving a smoother edge.

Methods of Antialiasing:

1. **Supersampling (Postfiltering):** The image is rendered at a much **higher resolution** than the display, then **averaged down (downsampled)** to the actual screen resolution — the averaging blends edge colors smoothly.
2. **Area Sampling (Prefiltering):** Calculates the exact geometric area of overlap between the pixel and the object's boundary, and sets pixel intensity based on that overlap percentage directly (more mathematically accurate but computationally expensive).
3. **Pixel Phasing:** Physically/electronically shifts the display position of pixels slightly to be closer to the true line path (used in some specialized hardware).

Advantages of Antialiasing

- Produces **smoother, more visually pleasing images** (especially for text and diagonal/curved lines).
- Improves the overall quality of rendered graphics and games.

Disadvantages of Antialiasing

- Requires **more computation/processing power** (especially supersampling).
- Can sometimes make edges look slightly **blurry** if overused.

Applications

- Used in video games, digital art software, font rendering (making on-screen text look smooth), CAD software.

Exam Tip: “What is Aliasing? Explain Antialiasing techniques” — common 5-mark question. Always explain WHY aliasing happens (discrete pixel grid vs continuous line) before explaining the fix.

Dithering

Definition

Dithering is a technique used to create the **illusion of more colors or shades** than are actually available on a display device, by arranging available colors in a **specific pattern** so that, when viewed from a normal distance, the human eye blends them together to perceive an intermediate color/shade that doesn't actually exist in the display's limited palette.

Zero-Level Explanation: Imagine you only have black and white dots (no gray). If you print black dots very close together in a pattern with white spaces between them, from a distance, your eye sees “gray” — even though there's technically no gray ink used at all! This is exactly how newspaper photographs (black & white) appear to have many shades of gray. That's dithering.

How it Works:

- A pattern (dither matrix) of available colors is used to approximate the desired unavailable color.
- Commonly used when the display or output device has a **limited color palette** (e.g., old monitors with only 256 colors, or black & white printers).

Types of Dithering:

1. **Ordered Dithering:** Uses a fixed, predefined pattern/matrix to decide pixel color combinations.
2. **Random Dithering:** Adds a small amount of random “noise” to pixel values before quantizing (reduces visible color-banding patterns).
3. **Error Diffusion Dithering (e.g., Floyd-Steinberg):** The “rounding error” from approximating one pixel’s color is carried forward (diffused) to neighboring pixels, so the overall average color remains accurate.

Advantages

- Allows devices with limited color palettes to display images that **appear to have more colors/shades** than they actually support.
- Reduces visible “color banding” (harsh, unnatural transitions between colors).

Disadvantages

- Can introduce a **grainy or “noisy” texture** to the image if the dither pattern is too coarse/visible.
- Not a substitute for actual higher color depth — it’s an *approximation* trick.

Applications

- Old computer displays/printers with limited colors.
- Image compression and printing (newspapers, black & white printers).

Memory Trick: Dithering = “**Faking more colors using patterns of fewer colors — like pointillism painting!**”

4.2 2-D Graphics Transformations

Introduction

Definition: A **Transformation** in computer graphics is a mathematical operation applied to an object (point, line, shape) to **change its position, size, orientation, or shape** on the screen.

Why are Transformations needed?

- To move, resize, rotate, or reshape objects without having to redraw them from scratch.
- Essential for animation, CAD design changes, and building complex scenes from simple objects.

Types of 2-D Transformations:

1. **Translation** – moving an object from one position to another.
2. **Rotation** – rotating an object around a fixed point (pivot).
3. **Scaling** – changing the size of an object (enlarging or shrinking).
4. **Reflection** – creating a mirror image of an object.
5. **Shearing** – slanting/distorting the shape of an object.

1. Translation

Definition: Translation moves every point of an object by the **same distance** in a given direction — the object's shape and size remain unchanged; only its position changes.

Formula:

$$\begin{aligned}x' &= x + tx \\ y' &= y + ty\end{aligned}$$

Where (tx, ty) is the Translation Vector (distance moved along x and y axes)

Matrix Form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

(Using **Homogeneous Coordinates** — a 3rd coordinate “1” is added to allow translation to be expressed as matrix multiplication, just like rotation/scaling.)

Solved Numerical Example — Translation

Translate a point $P(4, 5)$ by $tx=3, ty=2$.

$$\begin{aligned}x' &= 4 + 3 = 7 \\ y' &= 5 + 2 = 7\end{aligned}$$

New Point: $P'(7, 7)$

2. Rotation

Definition: Rotation turns/rotates an object about a fixed reference point (called the **pivot point**) by a specified **angle (θ)**.

Formula (rotation about origin, counter-clockwise by angle θ):

$$\begin{aligned}x' &= x \cdot \cos(\theta) - y \cdot \sin(\theta) \\ y' &= x \cdot \sin(\theta) + y \cdot \cos(\theta)\end{aligned}$$

Matrix Form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Solved Numerical Example — Rotation

Rotate a point $P(5, 5)$ by 90° about the origin (counter-clockwise).

$$\begin{aligned}\cos 90^\circ &= 0, \quad \sin 90^\circ = 1 \\ x' &= 5 \cdot (0) - 5 \cdot (1) = -5 \\ y' &= 5 \cdot (1) + 5 \cdot (0) = 5\end{aligned}$$

New Point: P'(-5, 5)

3. Scaling

Definition: Scaling changes the **size** of an object by multiplying the coordinate values by scaling factors **sx** (along x-axis) and **sy** (along y-axis).

Formula:

$$\begin{aligned}x' &= x * sx \\ y' &= y * sy\end{aligned}$$

Matrix Form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- If $sx = sy$ → **Uniform Scaling** (shape preserved, only size changes).
- If $sx \neq sy$ → **Differential/Non-Uniform Scaling** (shape distorted).
- If $sx, sy > 1$ → object enlarges (**magnification**).
- If $sx, sy < 1$ (between 0 and 1) → object shrinks (**minification**).

Solved Numerical Example — Scaling

Scale a point P(3, 4) with $sx=2$, $sy=3$.

$$\begin{aligned}x' &= 3*2 = 6 \\ y' &= 4*3 = 12\end{aligned}$$

New Point: P'(6, 12)

Important Note: Basic scaling scales relative to the **origin (0,0)** — this also shifts the object's position if it's not centered at origin. To scale about a specific fixed point, you must first translate the object so that fixed point becomes the origin, then scale, then translate back.

4. Reflection

Definition: Reflection produces a **mirror image** of an object about a given axis or line (like a mirror reflection).

Common Reflection Formulas:

Reflection About	Formula	Matrix
X-axis	$x'=x, y'=-y$	$[1,0,0 / 0,-1,0 / 0,0,1]$
Y-axis	$x'=-x, y'=y$	$[-1,0,0 / 0,1,0 / 0,0,1]$
Origin	$x'=-x, y'=-y$	$[-1,0,0 / 0,-1,0 / 0,0,1]$
Line $y=x$	$x'=y, y'=x$	$[0,1,0 / 1,0,0 / 0,0,1]$

Solved Numerical Example — Reflection

Reflect point P(3, 5) about the X-axis.

$$\begin{aligned}x' &= 3 \\y' &= -5\end{aligned}$$

New Point: P'(3, -5)

5. Shearing

Definition: Shearing distorts the shape of an object such that it appears “slanted” or “skewed” — like pushing the top of a rectangle sideways to make it a parallelogram.

Formula: - **X-Shear** (shifts x-coordinates based on y-value): $x' = x + shx*y$, $y' = y$ - **Y-Shear** (shifts y-coordinates based on x-value): $x' = x$, $y' = y + shy*x$

Matrix Form (X-Shear):

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Solved Numerical Example — X-Shear

Shear point P(2, 3) with $shx = 2$ (X-direction shear).

$$\begin{aligned}x' &= 2 + 2*3 = 8 \\y' &= 3\end{aligned}$$

New Point: P'(8, 3)

Memory Trick for all 5 transformations: “T-R-S-R-S” → Translation, Rotation, Scaling, Reflection, Shearing. Or remember: “**Move, Turn, Resize, Mirror, Slant.**”

Geometric and Coordinate Transformations

Geometric Transformation

Definition: A **Geometric Transformation** changes the **position, size, or orientation of an object itself**, while the **coordinate system stays fixed**. (This is what we covered above: Translation, Rotation, Scaling, etc.)

Coordinate Transformation

Definition: A **Coordinate Transformation**, on the other hand, keeps the **object fixed** but changes the **coordinate system/reference frame** used to describe it — essentially viewing the same object from a “different reference frame’s point of view.”

Key Difference: Moving the object 5 units right (Geometric) produces the same *visual* final result as moving the coordinate axes 5 units left while the object stays still (Coordinate) — they are mathematically related but conceptually different perspectives.

Basis	Geometric Transformation	Coordinate Transformation
What changes?	The object itself moves/changes	The reference frame/axes change
Object position	Changes in the fixed coordinate system	Object stays fixed; frame moves
Common use	Animating/editing objects	Changing viewpoint/reference frame (e.g., in 3D viewing)

Transformation Composition (Combined Transformations)

Definition

Transformation Composition refers to applying **multiple transformations in sequence** (e.g., first translate, then rotate, then scale) by **multiplying their individual transformation matrices together** to get a single **combined (composite) transformation matrix** — this is much more efficient than applying each transformation one at a time to every point.

Why is it needed?

- Complex transformations (like “rotate an object about a point other than the origin”) require a **sequence** of basic transformations:
 1. Translate the object so the pivot point moves to the origin.
 2. Rotate about the origin.
 3. Translate back to the original pivot point position.

Formula (Composite Matrix)

If T1, T2, T3 are individual transformation matrices applied in order, the composite matrix is:

Composite Matrix $M = T3 \times T2 \times T1$

New Point $[x' \ y' \ 1]^T = M \times [x \ y \ 1]^T$

IMPORTANT: Order Matters! Matrix multiplication is **not commutative** — applying Rotation then Translation gives a **different result** than applying Translation then Rotation. Always apply matrices in the correct order (rightmost matrix is applied first to the point).

Solved Numerical Example — Rotation about an arbitrary pivot point

Rotate point P(5,5) by 90° about pivot point Q(2,2) (not the origin).

Steps:

Step 1: Translate so Q moves to origin: $(x-2, y-2) \rightarrow P$ becomes (3,3)

Step 2: Rotate by 90°: $x' = x \cdot \cos 90^\circ - y \cdot \sin 90^\circ = 3 \cdot 0 - 3 \cdot 1 = -3$
 $y' = x \cdot \sin 90^\circ + y \cdot \cos 90^\circ = 3 \cdot 1 + 3 \cdot 0 = 3$
 $\rightarrow$ Rotated point: (-3, 3)

Step 3: Translate back by adding Q: $(-3+2, 3+2) = (-1, 5)$

Final Answer: P'(-1, 5)

Exam Tip: “Rotate a point about an arbitrary/fixed point (not origin)” is a **very common numerical question** (10 marks) — always show all 3 steps (translate to origin → rotate → translate back) clearly.

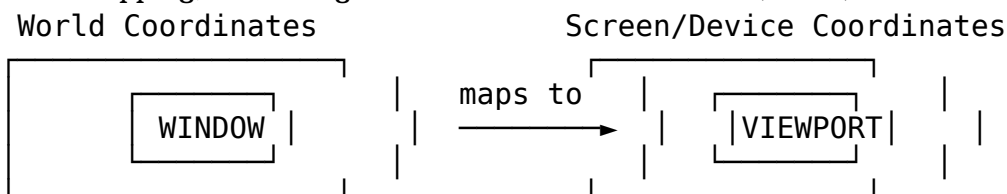
4.3 2-D Viewing

Definition

2-D Viewing is the process of displaying a selected part of a picture (defined in “world coordinates”) onto a display device (in “screen/device coordinates”). It involves defining a **Window** (the part of the scene we want to see) and a **Viewport** (the area on the screen where it will be displayed).

Key Terms:

- **World Coordinate System:** The coordinate system in which the actual picture/scene is defined/modeled (can be very large, e.g., a map of a whole country).
- **Window:** A rectangular region in the world coordinate system that specifies **what part of the scene** is to be displayed (like a camera’s field of view).
- **Viewport:** A rectangular region on the actual **display screen** where the content of the window will be shown.
- **Viewing Transformation (Window-to-Viewport mapping):** The process of mapping/converting coordinates from the Window (world) to the Viewport (screen).



Clipping

Definition

Clipping is the process of **identifying and removing (cutting off) the portions of a picture that lie outside a specified region** (the “clip window”), so that only the visible part within that region is processed/displayed.

Zero-Level Explanation: Think of clipping like using scissors to cut a photograph to fit inside a picture frame — anything outside the frame’s boundary is simply cut away and not shown.

Why is Clipping needed?

- To avoid wasting processing time drawing objects (or parts of objects) that are outside the visible viewing area anyway.
- To prevent visual errors/glitches from objects that extend beyond the screen/window boundary.

Exterior and Interior Clipping

- **Interior Clipping:** The normal/common form of clipping — retains (keeps) only the portion of the object that is **inside** the clip window, and discards everything outside it. (This is what “clipping” usually refers to by default.)
- **Exterior Clipping:** The reverse — retains (keeps) only the portion of the object that is **outside** the clip window, and discards the part inside it. Used in special cases, such as when one picture partially overlaps/overlays another, and you need to see only the non-overlapping (exterior) part.

Memory Trick: Interior = “keep the inside” (normal case). Exterior = “keep the outside” (special/rare case, e.g., overlapping picture management).

Line Clipping (Cohen-Sutherland Algorithm — commonly expected)

Theory

The **Cohen-Sutherland Line Clipping Algorithm** is a fast method to clip a line against a rectangular clipping window. It divides the entire 2D space into **9 regions** using a **4-bit region code** for each endpoint, based on the endpoint’s position relative to the window’s four boundaries (top, bottom, left, right).

Region Codes (4 bits: Top, Bottom, Right, Left)

1001		1000		1010	
0001		0000		0010	← 0000 = INSIDE the window
0101		0100		0110	

Each bit means: - Bit 1 (leftmost) = Above window (Top) - Bit 2 = Below window (Bottom) - Bit 3 = Right of window - Bit 4 (rightmost) = Left of window

Algorithm Steps:

1. Calculate the 4-bit region code for both endpoints of the line.
2. If both codes are 0000 (both points inside) → TRIVIAL ACCEPT, draw the line as is.
3. If the bitwise AND of both codes is NOT 0000 (both points share an outside region) → TRIVIAL REJECT, the line is completely outside, discard it.
4. Otherwise (line may be partially inside):
 - Pick a point with a non-zero code (outside the window).
 - Find the intersection of the line with the corresponding window boundary.
 - Replace that endpoint with the intersection point.
 - Repeat from Step 1 until trivial accept or reject occurs.

Advantages

- Very efficient for lines that are entirely inside or entirely outside the window (fast trivial accept/reject using simple bitwise operations).

Disadvantages

- Requires multiple iterations (intersection calculations) for lines that cross multiple boundaries.

Exam Tip: “Explain Cohen-Sutherland Line Clipping Algorithm with region codes” is a **frequently asked 10-mark question** — always draw the 9-region diagram with 4-bit codes.

Viewport Transformation

Definition

Viewport Transformation is the mathematical mapping that converts coordinates from the **Window (world coordinates)** to the **Viewport (screen/device coordinates)**, so that the visible part of the scene is correctly scaled and positioned on the actual display.

Formula

Given Window with corners (xw_min, yw_min) to (xw_max, yw_max), and Viewport with corners (xv_min, yv_min) to (xv_max, yv_max):

Scale factor $S_x = (xv_max - xv_min) / (xw_max - xw_min)$

Scale factor $S_y = (yv_max - yv_min) / (yw_max - yw_min)$

$xv = xv_min + (xw - xw_min) * S_x$

$yv = yv_min + (yw - yw_min) * S_y$

Solved Numerical Example — Viewport Transformation

Window: (0,0) to (10,10). Viewport: (0,0) to (100,100). Map point (5,5).

$S_x = (100-0)/(10-0) = 10$

$S_y = (100-0)/(10-0) = 10$

$xv = 0 + (5-0)*10 = 50$

$yv = 0 + (5-0)*10 = 50$

New Point: (50, 50)

Polygon Clipping (Sutherland-Hodgman Algorithm)

Theory

The **Sutherland-Hodgman Algorithm** clips a **polygon** (not just a single line) against a rectangular (or convex) clipping window. It processes the polygon **edge by edge, clipping against one window boundary at a time** (left, right, top, bottom), producing a new (possibly larger) list of vertices after each boundary is processed.

Algorithm Steps (for one clip boundary):

For each edge (from vertex V1 to V2) of the polygon:

1. If V1 is inside and V2 is inside → output V2.
2. If V1 is inside and V2 is outside → output the intersection point

- (where the edge crosses the boundary).
3. If V1 is outside and V2 is inside → output both the intersection point AND V2.
 4. If V1 is outside and V2 is outside → output nothing.

Repeat this process sequentially for all 4 boundaries (left, right, top, bottom) of the clip window, using the output of one boundary as the input for the next.

Advantages

- Works correctly for clipping **polygons** (not just individual lines) against a convex window.
- Simple boundary-by-boundary processing.

Disadvantages

- Can produce extra/unwanted “phantom” edges when clipping **concave** polygons against certain windows (needs care/extension for concave cases).

Exam Tip: “Explain Sutherland-Hodgman Polygon Clipping Algorithm” — common 10-mark question; be ready to explain the 4 cases (in-in, in-out, out-in, out-out) clearly with a diagram.

Text Clipping

Definition

Text Clipping deals specifically with clipping (cutting off) text/characters that fall partially or fully outside the clipping window.

Methods of Text Clipping:

1. **All-or-None (String) Clipping:** The simplest method — if the **entire text string’s bounding box** is not completely inside the clip window, the **whole string is rejected** (not displayed at all). If it’s completely inside, it’s displayed as-is.
2. **All-or-None (Character) Clipping:** A slightly more refined method — clipping is checked **character by character** instead of the whole string. Each individual character is either fully displayed (if entirely inside the window) or fully discarded (if not) — this way, at least the characters that do fit are shown.
3. **Exact (Component) Clipping:** The most precise (but most computationally expensive) method — treats each character as a graphic object made of lines/curves, and clips it exactly at the pixel/component level (partial characters can be shown, cut exactly at the boundary), just like polygon or line clipping.

Comparison Table: Text Clipping Methods

Method	Precision	Speed	Use Case
All-or-None String	Lowest	Fastest	Simple applications, low precision needed
All-or-None	Medium	Medium	Balanced approach

Method	Precision	Speed	Use Case
Character			— common default
Exact/Component	Highest	Slowest	High-precision graphics/CAD software

Memory Trick: Text Clipping precision increases as: **String** → **Character** → **Component** (like zooming in from whole sentence, to whole word, to exact letter shape).

IMPORTANT POINTS TO REMEMBER (UNIT IV QUICK RECAP)

- **Aliasing** = jagged edges due to pixel grid; **Antialiasing** = smoothing technique (supersampling, area sampling).
- **Dithering** = simulates more colors using patterns of available colors.
- **5 Basic 2D Transformations:** Translation (move), Rotation (turn), Scaling (resize), Reflection (mirror), Shearing (slant) — all expressed using homogeneous coordinate matrices.
- **Transformation Composition** = combining multiple transformations via matrix multiplication; order matters!
- **Rotation about an arbitrary point** = Translate to origin → Rotate → Translate back (3-step process, common numerical).
- **Window** = part of world coordinates to view; **Viewport** = screen area to display it; **Viewport Transformation** maps one to the other.
- **Clipping** = removing parts of a picture outside the clip window. **Interior** clipping keeps inside; **Exterior** keeps outside.
- **Cohen-Sutherland** = line clipping using 4-bit region codes (9 regions).
- **Sutherland-Hodgman** = polygon clipping, edge-by-edge, boundary-by-boundary.
- **Text Clipping:** All-or-None String < All-or-None Character < Exact/Component (increasing precision).

CHAPTER-END QUESTIONS — UNIT IV

20 Short Questions

1. Define Aliasing.
2. Define Antialiasing.
3. Name any two methods of Antialiasing.
4. Define Dithering.
5. What is Translation? Give its formula.
6. What is Rotation? Give its formula.
7. What is Scaling? Differentiate uniform and non-uniform scaling.
8. What is Reflection?
9. What is Shearing?
10. Differentiate Geometric and Coordinate transformation.
11. What is Transformation Composition?

12. Define Window and Viewport.
13. What is Clipping?
14. Differentiate Interior and Exterior Clipping.
15. What is a region code in Cohen-Sutherland Algorithm?
16. What is trivial accept and trivial reject?
17. Name the algorithm used for polygon clipping.
18. What is Viewport Transformation?
19. Define Text Clipping.
20. Name the three methods of text clipping.

15 Long Questions

1. Explain Aliasing and Antialiasing techniques in detail.
2. Explain Dithering with its types and applications.
3. Explain 2D Transformations (Translation, Rotation, Scaling) with matrices and examples.
4. Explain Reflection and Shearing with formulas and examples.
5. Explain Transformation Composition with a solved numerical (rotation about arbitrary point).
6. Differentiate Geometric and Coordinate Transformation.
7. Explain the concept of Window and Viewport with a diagram.
8. Explain Viewport Transformation with a solved numerical.
9. Explain Cohen-Sutherland Line Clipping Algorithm with region codes and diagram.
10. Explain Sutherland-Hodgman Polygon Clipping Algorithm with steps.
11. Explain Interior and Exterior Clipping with examples.
12. Explain the three methods of Text Clipping.
13. Solve a numerical: Rotate a given point about an arbitrary pivot point.
14. Solve a numerical: Scale a given object and find new coordinates.
15. Explain why the order of transformations matters in composition, with an example.

10 Viva Questions

1. What causes aliasing on a raster display?
2. Name one antialiasing technique.
3. What is the formula for translation?
4. What is homogeneous coordinate system used for?
5. What are the two scaling factors called?
6. What is a pivot point in rotation?
7. What is a clip window?
8. How many regions does Cohen-Sutherland algorithm divide the plane into?
9. What is the difference between window and viewport?
10. Which clipping algorithm is used for polygons?

MCQs with Answers

1. Aliasing occurs due to:

- (a) Too many colors (b) Discrete pixel grid (c) Slow refresh rate (d) Large screen size — **Answer: (b) Discrete pixel grid**
2. Dithering is used to:
 (a) Speed up rendering (b) Simulate more colors using patterns (c) Clip images (d) Rotate objects — **Answer: (b) Simulate more colors using patterns**
3. Translation changes the object's:
 (a) Size (b) Position (c) Shape (d) Color — **Answer: (b) Position**
4. In Cohen-Sutherland algorithm, a region code of 0000 means:
 (a) Outside window (b) Inside window (c) On the boundary (d) Undefined — **Answer: (b) Inside window**
5. Sutherland-Hodgman algorithm is used for:
 (a) Line clipping (b) Polygon clipping (c) Text clipping (d) Circle clipping — **Answer: (b) Polygon clipping**
6. Homogeneous coordinates add which extra value?
 (a) 0 (b) -1 (c) 1 (d) 2 — **Answer: (c) 1**
7. If $s_x = s_y$ in scaling, it is called:
 (a) Differential scaling (b) Uniform scaling (c) Reflection (d) Shearing — **Answer: (b) Uniform scaling**
8. Exterior clipping keeps the part of the object that is:
 (a) Inside the window (b) Outside the window (c) On the boundary (d) None of these — **Answer: (b) Outside the window**
9. Which text clipping method is most precise?
 (a) All-or-None String (b) All-or-None Character (c) Exact/Component (d) None — **Answer: (c) Exact/Component**
10. Viewport refers to:
 (a) Part of world coordinates selected (b) Area on screen where content is displayed (c) A clipping algorithm (d) A transformation matrix — **Answer: (b) Area on screen where content is displayed**

Previous-Year Style Questions (BRABU Pattern)

- Explain 2D Transformations with matrices and examples. (10 marks — very frequently asked)
- Explain Cohen-Sutherland Line Clipping Algorithm. (10 marks — guaranteed almost every year)
- What is Antialiasing? Explain its techniques. (5 marks)
- Explain Window to Viewport transformation with numerical. (5/10 marks)
- Differentiate Interior and Exterior Clipping. (2/5 marks)
- Solve: Rotate point P about a given pivot point (not origin). (10 marks — very common numerical)

UNIT V — 3D GRAPHICS, PROJECTIONS & CURVES

5.1 Introduction to 3-D Graphics

Definition: 3-D (Three-Dimensional) Computer Graphics deals with representing, transforming, and displaying objects that have **three dimensions** — **width (x)**, **height (y)**, **and depth (z)** — giving a realistic sense of depth, unlike 2D graphics which only has width and height.

Why 3-D Graphics?

- Real-world objects have depth — 3D graphics allows realistic modeling of real objects (buildings, cars, human faces, etc.).
- Used extensively in movies, games, CAD/engineering design, medical imaging, and virtual/augmented reality.

3-D Graphics Transformations

Just like 2D transformations, 3D transformations move/resize/rotate objects, but now involve an additional **z-coordinate** and use **4×4 matrices** (instead of 3×3) with homogeneous coordinates (x, y, z, 1).

1. 3-D Translation

Formula:

$$\begin{aligned}x' &= x + tx \\y' &= y + ty \\z' &= z + tz\end{aligned}$$

Matrix:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Solved Example — 3D Translation

Translate point P(2,3,4) by (tx,ty,tz) = (1,2,3).

$$\begin{aligned}x' &= 2+1 = 3 \\y' &= 3+2 = 5 \\z' &= 4+3 = 7\end{aligned}$$

New Point: P'(3, 5, 7)

2. 3-D Scaling

Formula:

$$x' = x*s_x, \quad y' = y*s_y, \quad z' = z*s_z$$

Matrix:

$$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3. 3-D Rotation

Unlike 2D (which has only 1 axis of rotation — around a point), 3D rotation can occur around **three different axes**: X-axis, Y-axis, or Z-axis.

Rotation about Z-axis (same as 2D rotation, with z unchanged):

$$\begin{aligned} x' &= x \cos\theta - y \sin\theta \\ y' &= x \sin\theta + y \cos\theta \\ z' &= z \end{aligned}$$

Rotation about X-axis:

$$\begin{aligned} y' &= y \cos\theta - z \sin\theta \\ z' &= y \sin\theta + z \cos\theta \\ x' &= x \end{aligned}$$

Rotation about Y-axis:

$$\begin{aligned} z' &= z \cos\theta - x \sin\theta \\ x' &= z \sin\theta + x \cos\theta \\ y' &= y \end{aligned}$$

Memory Trick: For rotation about any axis, that axis's coordinate **stays unchanged**, and the other two coordinates follow the same “cos/sin” pattern as 2D rotation, cyclically: **X→Y→Z→X**.

5.2 Coordinate Transformations (3D)

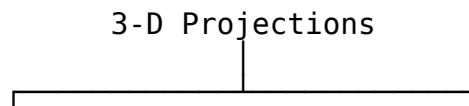
Similar to 2D, a **Coordinate Transformation** in 3D changes the reference frame/axes (viewpoint) while the object stays fixed — commonly used to convert from **World Coordinates** to **Viewing/Camera Coordinates** (i.e., transforming the scene as if viewed from a specific camera position and direction in 3D space).

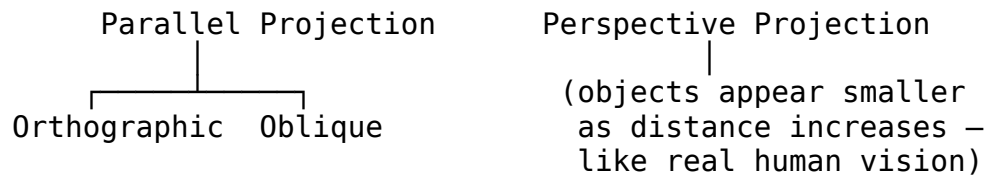
3-D Projections

Definition

Projection is the process of converting/mapping a 3-D object (which has x, y, z coordinates) onto a **2-D plane** (like a computer screen, which only has x, y) — because all display devices are ultimately flat (2D) surfaces.

Types of Projections:





Perspective Projection

Definition

Perspective Projection is a projection technique where **parallel lines appear to converge at a point** (called the **Center of Projection, COP**), and objects appear **smaller as their distance from the viewer increases** — this mimics how the **human eye/camera** actually sees the real world, giving a **realistic sense of depth**.

Zero-Level Explanation: Think about looking down a long straight railway track — the two rails appear to meet at a point far in the distance, even though they’re actually parallel in real life. This “vanishing point” effect is exactly what perspective projection recreates on a 2D screen.

Perspective Projection on Plane with Center of Projection C(0,0,0)

Setup: Center of Projection (COP) is at the origin C(0,0,0), and the projection plane is at $z = d$ (some distance d along the z -axis).

Formula (derived using similar triangles):

$$\begin{aligned} x_p &= (x * d) / z \\ y_p &= (y * d) / z \\ z_p &= d \quad (\text{fixed, since it lies on the projection plane}) \end{aligned}$$

Where (x, y, z) is the original 3D point, and (x_p, y_p, z_p) is its projected position on the plane $z=d$.

Solved Numerical Example

Project point P(6, 9, 3) onto the plane $z=d=1$, with COP at origin (0,0,0).

$$\begin{aligned} x_p &= (6 * 1) / 3 = 2 \\ y_p &= (9 * 1) / 3 = 3 \\ z_p &= 1 \end{aligned}$$

Projected Point: P'(2, 3, 1)

Perspective Projection on Plane with Center of Projection C(a, b, c) (General Case)

Setup: Now the Center of Projection is at an arbitrary point C(a, b, c), not necessarily the origin.

Method: The projection is calculated by finding the intersection of the line joining the Center of Projection C(a,b,c) to the point P(x,y,z), with the projection plane (commonly $z=0$ or another chosen plane).

Formula (for projection plane $z=0$, general COP at (a,b,c)):

Using parametric line equation from C to P, solving for where $z=0$:

$$\begin{aligned}x_p &= a + (x - a) * [(0 - c) / (z - c)] \\y_p &= b + (y - b) * [(0 - c) / (z - c)] \\z_p &= 0\end{aligned}$$

Key Concept: This is a more general version of the origin-COP case — when $C = (0,0,0)$, this formula simplifies back to the earlier simpler formula.

Advantages of Perspective Projection

- Produces **realistic images** (matches human visual perception) — used in movies, games, architectural walkthroughs.
- Gives a clear sense of **depth and distance**.

Disadvantages of Perspective Projection

- Does **not preserve actual size/proportions** of objects (a far object looks smaller than it actually is) — not suitable for precise engineering measurement drawings.
- More complex mathematics compared to parallel projection.

Applications

- Video games, movies/animation, architectural walkthroughs/visualization, virtual reality.

Exam Tip: Both perspective projection formulas (COP at origin, and COP at general point) are **frequently asked as 10-mark numerical questions** — practice both formula derivations and numericals.

Parallel Projections

Definition

Parallel Projection is a projection technique where **projection lines (rays) are parallel to each other** (they do not converge to a point) — this means the **size and proportions of the object are preserved** regardless of distance from the viewer, though it looks less “realistic” than perspective projection.

Types of Parallel Projection:

1. **Orthographic Projection:** Projection lines are **perpendicular (90°)** to the projection plane.
 - **Multi-view Orthographic:** Shows separate views from the front, top, and side (commonly used in engineering drawings/blueprints).
 - **Axometric Projection (e.g., Isometric):** A single view showing 3 sides of an object at once, using equal angles, giving a 3D-like appearance while still being a parallel projection.

2. **Oblique Projection:** Projection lines are **NOT perpendicular** to the projection plane — they meet the plane at an angle other than 90°, allowing one face of the object to be shown in true shape/size while giving a sense of depth for the other faces.
 - **Cavalier Projection:** Depth lines are drawn at **full (true) scale**.
 - **Cabinet Projection:** Depth lines are drawn at **half scale** ($\frac{1}{2}$ the true length), giving a more realistic appearance than Cavalier.

Perspective vs Parallel Projection — Comparison Table (VERY IMPORTANT)

Basis	Perspective Projection	Parallel Projection
Projection Lines	Converge at Center of Projection (COP)	Remain parallel (never converge)
Realism	High (matches human eye)	Lower (looks more “flat”/technical)
Size/Proportion	NOT preserved (distant objects look smaller)	Preserved (true size maintained)
Use Case	Games, movies, VR	Engineering drawings, CAD blueprints
Sub-types	1-point, 2-point, 3-point perspective	Orthographic, Oblique

Memory Trick: “Perspective = Realistic but distorts size (like a photo); Parallel = Accurate size but looks technical (like a blueprint).”

Exam Tip: “Differentiate Perspective and Parallel Projection” is **almost guaranteed** every year at BRABU (5 marks) — memorize this table.

3-D Viewing

Definition: 3-D Viewing is the process of specifying **what part of a 3D scene** should be visible, and **from which viewpoint (camera position)**, and then projecting that visible part onto the 2D screen — analogous to 2D Window-to-Viewport viewing, but extended to three dimensions with an added **View Volume** (a 3D region, like a pyramid or box, defining visible space).

Key Steps in 3-D Viewing Pipeline:

1. Define the **View Plane** (where the image will be projected).
2. Define the **View Volume** (the visible region of 3D space — shaped like a frustum for perspective, or a box for parallel projection).
3. Perform **3D Clipping** (remove parts of objects outside the view volume).
4. Perform **Projection** (convert remaining 3D coordinates to 2D).
5. Map to the **Viewport** on the actual screen.

Clipping (3D)

Definition: Similar to 2D clipping, **3D Clipping** removes the parts of 3D objects that lie **outside the View Volume** — but now clipping must be checked against **6 boundary planes** instead of 4 boundary lines (left, right, top, bottom, **near**, and **far** planes).

Hidden Lines and Hidden Surfaces / Scan Line Entries

Definition

Hidden Line/Surface Removal (HLSR) refers to algorithms/techniques used to determine which lines/surfaces of a 3D object are **visible** to the viewer and which are **hidden (blocked)** behind other surfaces of the same object — and removing/not drawing the hidden ones, so the displayed image looks correct and realistic (just like in real life, you cannot see the back side of a solid object).

Zero-Level Explanation: If you look at a solid cube from one angle, you can only see 3 of its 6 faces — the other 3 faces are “hidden” behind the visible ones. If a computer draws ALL 6 faces without hiding logic, the image looks confusing/wrong (like an X-ray). Hidden Surface Removal fixes this by only drawing what should actually be visible.

Common Methods:

1. **Z-Buffer (Depth-Buffer) Algorithm:** Maintains a “depth value” (distance from viewer) for every pixel; only the closest surface’s color at each pixel is kept/displayed (simple, widely used in modern graphics hardware).
2. **Scan-Line Algorithm (for hidden surfaces):** Processes the image **row by row (scan line by scan line)** — similar in spirit to scan-line polygon filling — determining, for each scan line, which surface is nearest to the viewer at each pixel position along that row, and displaying only that surface’s color. This is sometimes referred to as using “**scan line entries**” — the list of edge/surface intersections recorded for each scan line to determine visibility and fill order.
3. **Painter’s Algorithm:** Sorts all surfaces by their distance from the viewer (farthest to nearest) and draws them in that order — nearer surfaces are drawn last, naturally covering/hiding farther surfaces (like a painter painting background first, then foreground on top).
4. **Back-Face Detection/Culling:** A quick, simple check — for objects like a sphere or cube where the back faces are guaranteed to be hidden by front faces, the algorithm calculates the surface normal (direction the face is pointing) and immediately discards (“culls”) faces pointing away from the viewer, without further processing.

Comparison Table: Hidden Surface Removal Methods

Method	Basis	Speed	Notes
Z-Buffer	Depth value per pixel	Fast (hardware-supported)	Most widely used in modern GPUs
Scan-Line	Row-by-row visibility determination	Moderate	Good for polygon-based scenes
Painter’s Algorithm	Sort surfaces back-to-front	Moderate	Fails for intersecting/overlapping surfaces
Back-Face Culling	Surface normal direction	Very Fast	Only removes obviously-hidden

Method	Basis	Speed	Notes
			back faces; not a complete solution alone

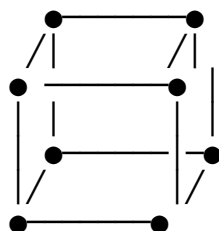
Exam Tip: “Explain Hidden Surface/Line removal algorithms” is a **frequently asked 10-mark question** — mention at least Z-buffer and Painter’s Algorithm with their working.

5.3 Importance of Wireframe Models

Definition

A **Wireframe Model** is a simple 3D representation of an object using only its **edges and vertices (corner points)** — drawn as a skeleton/framework of connected lines — **without any surface, color, or shading** (you can see “through” the object, as if it’s made of wire).

Wireframe Cube:



Importance/Advantages of Wireframe Models:

- **Simple and fast to compute/render** — since there’s no surface shading/color calculation needed.
- Useful in the **early design/drafting stages** to quickly visualize the basic structure/shape of an object.
- Requires **very little memory/storage**.
- Easy to view the **internal structure** of an object (since it’s “see-through”).

Demerits (Disadvantages) of Wireframe Models:

- **Not realistic** — cannot show solid surfaces, shading, lighting, texture, or color.
- **Ambiguous depth perception** — it can be hard to tell which edges are in front and which are behind (without hidden-line removal, wireframes can look confusing, e.g., the classic “Necker Cube” illusion).
- Not suitable for final, presentation-quality, photorealistic rendering.

Applications

- Early-stage CAD/engineering design drafts.
- 3D modeling software’s “preview/draft” display mode (before applying full rendering).

Representing Polygonal Models (Surface/Solid Models)

Definition

A **Polygonal Model** represents a 3D object's surface using a collection of connected **flat polygons** (usually triangles or quadrilaterals) that approximate the object's shape — each polygon has defined vertices, edges, and a surface that can be shaded/colored/textured, unlike a wireframe.

How it works:

- The surface of a complex 3D object (like a human face or a car body) is broken down (approximated) into many small, flat polygon “patches” — the more polygons used, the smoother/more detailed and realistic the model appears (this is called **polygon count** or **poly count**).
- Common data structures: **Vertex List** (coordinates of all corner points) + **Edge List** (connections between vertices) + **Face/Polygon List** (which vertices form each polygon face).

Advantages

- Can represent **realistic, solid, shaded 3D objects** (unlike wireframe).
- Widely supported by graphics hardware (GPUs are optimized for polygon/triangle rendering).

Disadvantages

- Curved surfaces are only an **approximation** (made of many small flat polygons) — true smooth curves require a very high polygon count (more memory/processing).
- More memory and processing power needed compared to simple wireframe models.

Memory Trick: Wireframe = “skeleton only” (edges); Polygonal Model = “skin added” (surfaces on the skeleton, can be shaded/colored).

Bezier Curves and Surfaces

Definition

A **Bezier Curve** is a smooth, mathematically-defined curve that is controlled by a set of **Control Points** — the curve doesn't necessarily pass through all control points (except the first and last), but its overall shape is “pulled” toward them, allowing designers to intuitively shape smooth curves by simply moving control points.

Zero-Level Explanation: Imagine you have a rubber band stretched between a few pins (control points) on a board. If you pull one pin, the rubber band bends smoothly toward it, but the band's start and end stay fixed at the first and last pins. That's essentially how a Bezier curve behaves — it's a smooth curve “guided” by control points.

Formula (General Bezier Curve, degree n , with $n+1$ control points $P_0...P_n$):

$$P(t) = \sum [P_i * B(i,n,t)] \quad \text{for } i = 0 \text{ to } n$$

Where $B(i,n,t)$ is the Bernstein Polynomial (Blending Function):
 $B(i,n,t) = C(n,i) * t^i * (1-t)^{(n-i)}$

t ranges from 0 to 1 (parameter along the curve)
 $C(n,i) = n! / (i!(n-i)!)$ [binomial coefficient]

Cubic Bezier Curve (most common — degree 3, 4 control points P_0, P_1, P_2, P_3)

$$P(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3, \quad 0 \leq t \leq 1$$

Properties of Bezier Curves:

1. The curve **always starts at P_0 and ends at P_n** (first and last control points).
2. The curve is **tangent to the first and last segments** of the control polygon (the line connecting the control points in order) — i.e., it starts moving in the direction of P_0-P_1 and ends moving in the direction of $P_{n-1}-P_n$.
3. The curve lies **entirely within the Convex Hull** of its control points (the smallest convex polygon that contains all the control points) — this is called the **Convex Hull Property**.
4. Moving any single control point affects the **shape of the entire curve** (global control/influence) — this is a limitation addressed by B-Splines (below).

Solved Numerical Example — Cubic Bezier

Given control points $P_0(0,0)$, $P_1(1,2)$, $P_2(3,3)$, $P_3(4,0)$. Find the point on the curve at $t=0.5$.

$$(1-t) = 0.5$$

$$(1-t)^3 = 0.125$$

$$3t(1-t)^2 = 3(0.5)(0.25) = 0.375$$

$$3t^2(1-t) = 3(0.25)(0.5) = 0.375$$

$$t^3 = 0.125$$

$$\begin{aligned} x &= 0.125(0) + 0.375(1) + 0.375(3) + 0.125(4) \\ &= 0 + 0.375 + 1.125 + 0.5 = 2.0 \end{aligned}$$

$$\begin{aligned} y &= 0.125(0) + 0.375(2) + 0.375(3) + 0.125(0) \\ &= 0 + 0.75 + 1.125 + 0 = 1.875 \end{aligned}$$

Point at $t=0.5$: $P(0.5) = (2.0, 1.875)$

Bezier Surfaces

- An extension of Bezier curves into **two parametric directions (u and v)** — instead of a set of control points forming a curve, a **grid/net of control points** forms a smooth surface patch.
- Used to model smooth, curved 3D surfaces (like car bodies, character skin in animation).

Advantages of Bezier Curves

- **Intuitive to design/control** — designers just move control points to reshape the curve.

- Smooth and mathematically well-behaved (guaranteed continuity).
- Widely supported in graphics/design software (fonts, vector graphics, animation paths).

Disadvantages of Bezier Curves

- **Global control problem** — moving ONE control point affects the ENTIRE curve's shape, making fine local adjustments difficult for complex curves.
- Higher-degree Bezier curves (many control points) become computationally expensive and harder to control intuitively.

Applications

- Font design (the curves of letters in fonts like TrueType/PostScript fonts are Bezier curves).
- Vector graphics software (Adobe Illustrator, CorelDRAW).
- Animation path design (motion paths in animation software).
- CAD/automobile body design.

Exam Tip: “Explain Bezier Curves with properties and a numerical example” is a **very frequently asked 10-mark question** — always mention all 4 properties (start/end at P_0/P_n , tangent property, convex hull, global control) and show the cubic Bezier formula with a solved example.

B-Splines

Definition

A **B-Spline (Basis Spline)** is an improved, more flexible version of the Bezier curve that solves the “global control” problem — a B-Spline curve is made up of **multiple polynomial curve segments** joined together smoothly, where each control point only influences a **local portion** of the curve (not the entire curve).

Key Differences from Bezier Curves:

1. **Local Control:** Moving one control point only affects a **small local region** of the curve, not the whole curve (unlike Bezier's global control) — this makes B-Splines much better for editing complex, detailed curves.
2. **Degree is Independent of Control Points:** In a Bezier curve, the degree of the curve is fixed by the number of control points ($n+1$ points = degree n curve). In a B-Spline, you can have **any number of control points** with a **fixed, chosen degree** (e.g., always cubic, degree 3), regardless of how many control points are used.
3. **Uses a “Knot Vector”:** A B-Spline requires an additional sequence of parameter values called a **Knot Vector**, which determines how the control points influence different sections of the curve.

Types of B-Splines:

- **Uniform B-Spline:** Knot values are evenly (uniformly) spaced.
- **Non-Uniform B-Spline:** Knot values are unevenly spaced — allows more flexible control (this leads to **NURBS — Non-Uniform Rational B-Splines**, widely used in professional 3D modeling software).

Bezier vs B-Spline — Comparison Table (VERY IMPORTANT)

Basis	Bezier Curve	B-Spline Curve
Control	Global (1 point affects whole curve)	Local (1 point affects only nearby section)
Degree vs Control Points	Degree = (number of control points - 1)	Degree is independent/fixed, regardless of control point count
Extra Requirement	None	Requires a Knot Vector
Flexibility for complex curves	Lower (hard to locally edit)	Higher (easy local editing)
Common Use	Fonts, simple vector paths	Complex 3D modeling (cars, characters), CAD

Memory Trick: “Bezier = one point, whole curve moves (global); B-Spline = one point, only nearby curve moves (local, like a ‘spring’ effect that fades out).”

Advantages of B-Splines

- **Local control** — much easier to edit specific sections of complex curves without disturbing the rest.
- More flexible — can add more control points without increasing the mathematical degree/complexity of the curve.

Disadvantages of B-Splines

- More **mathematically complex** than Bezier curves (knot vectors add complexity).
- Slightly harder for beginners to understand and control intuitively.

Applications

- Professional 3D modeling and animation software (Maya, Blender, 3ds Max).
- CAD systems for complex automobile/aircraft body design (often using NURBS specifically).

Exam Tip: “Differentiate Bezier Curve and B-Spline Curve” is a **very frequently asked 5-mark question** at BRABU — memorize the comparison table, especially the **Local vs Global control** difference, as this is the #1 differentiator examiners look for.

IMPORTANT POINTS TO REMEMBER (UNIT V QUICK RECAP)

- **3D Transformations** extend 2D ones using 4×4 matrices with an added z-coordinate; rotation can be about X, Y, or Z axis.
- **Projection** converts 3D → 2D. **Perspective Projection** (lines converge at COP, realistic, size not preserved) vs **Parallel Projection** (lines stay parallel, size preserved, used in engineering drawings).
- Perspective formulas: with COP at origin: $x_p = x_d/z$, $y_p = y_d/z$; with COP at C(a,b,c): uses the general line-plane intersection formula.
- **Parallel Projection types:** Orthographic (90° to plane) and Oblique (Cavalier=full scale, Cabinet=half scale depth).

- **Hidden Surface Removal** methods: Z-Buffer (most common, fast), Scan-Line, Painter's Algorithm, Back-Face Culling.
- **Wireframe Models** = edges/vertices only (fast, not realistic); **Polygonal Models** = surfaces added (realistic, more processing).
- **Bezier Curves** = smooth curve guided by control points; global control; passes through only first & last points; lies in convex hull.
- **B-Splines** = improved Bezier with **local control** (via knot vectors); degree independent of control point count.

CHAPTER-END QUESTIONS — UNIT V

20 Short Questions

1. Define 3D Computer Graphics.
2. Give the matrix form of 3D Translation.
3. What stays unchanged during rotation about the Z-axis?
4. Define Projection.
5. Define Perspective Projection.
6. Define Parallel Projection.
7. What is Center of Projection (COP)?
8. Name two types of Parallel Projection.
9. Differentiate Cavalier and Cabinet projection (one line).
10. What is Hidden Surface Removal?
11. Name any two Hidden Surface Removal algorithms.
12. Define Wireframe Model.
13. Give one demerit of Wireframe Models.
14. Define Polygonal Model.
15. Define Bezier Curve.
16. What is a Control Point?
17. What is the Convex Hull property of Bezier curves?
18. Define B-Spline.
19. What is a Knot Vector?
20. What is the main advantage of B-Spline over Bezier curve?

15 Long Questions

1. Explain 3D Transformations (Translation, Scaling, Rotation) with matrices.
2. Explain Perspective Projection with COP at origin, with a solved numerical.
3. Explain Perspective Projection with COP at C(a,b,c).
4. Differentiate Perspective and Parallel Projection with a table.
5. Explain types of Parallel Projection (Orthographic and Oblique) in detail.
6. Explain Hidden Line and Hidden Surface Removal algorithms.
7. Explain the Z-Buffer Algorithm for hidden surface removal.
8. Explain Painter's Algorithm for hidden surface removal.
9. Explain Wireframe Models — importance and demerits.

10. Explain Polygonal Models and how they differ from Wireframe Models.
11. Explain Bezier Curves with properties and a solved numerical example.
12. Explain B-Splines and differentiate them from Bezier Curves.
13. Explain 3D Viewing and the concept of View Volume.
14. Explain 3D Clipping and how it differs from 2D clipping.
15. Explain the applications of Bezier Curves and B-Splines in real software.

10 Viva Questions

1. How many axes can 3D rotation occur around?
2. What is the size of the transformation matrix used in 3D?
3. Which projection type is used in engineering drawings?
4. What does COP stand for?
5. Name one hardware-supported hidden surface algorithm.
6. What is a wireframe model also sometimes confused with visually (illusion)?
7. Does a Bezier curve pass through all its control points?
8. What is the degree of a cubic Bezier curve?
9. What gives B-Splines their local control property?
10. Name one software that uses NURBS.

MCQs with Answers

1. In 3D rotation about the Z-axis, which coordinate remains unchanged?
(a) x (b) y (c) z (d) None — **Answer: (c) z**
2. Perspective Projection lines:
(a) Remain parallel (b) Converge at a point (c) Never meet (d) Are curved —
Answer: (b) Converge at a point
3. Which projection preserves true size and shape?
(a) Perspective (b) Parallel (c) Both (d) Neither — **Answer: (b) Parallel**
4. Cabinet projection draws depth lines at:
(a) Full scale (b) Half scale (c) Double scale (d) Zero scale — **Answer: (b) Half scale**
5. Z-Buffer algorithm is used for:
(a) Color mixing (b) Hidden surface removal (c) Text clipping (d) Line drawing —
Answer: (b) Hidden surface removal
6. Wireframe models show only:
(a) Surfaces (b) Edges and vertices (c) Colors (d) Textures — **Answer: (b) Edges and vertices**
7. A cubic Bezier curve has how many control points?
(a) 2 (b) 3 (c) 4 (d) 5 — **Answer: (c) 4**
8. Bezier curve always passes through:
(a) All control points (b) Only the first control point (c) First and last control points
(d) No control points — **Answer: (c) First and last control points**
9. B-Splines use an additional structure called:

- (a) Convex Hull (b) Knot Vector (c) Bounding Box (d) Region Code — **Answer: (b) Knot Vector**

10. B-Splines provide which type of control?

- (a) Global (b) Local (c) No control (d) Random — **Answer: (b) Local**

Previous-Year Style Questions (BRABU Pattern)

- Explain Perspective Projection with COP at origin and at C(a,b,c) with numerical. (10 marks — frequently asked)
- Differentiate Perspective and Parallel Projection. (5 marks — very common)
- Explain Hidden Surface Removal algorithms. (10 marks)
- Explain Bezier Curve with properties and numerical example. (10 marks — guaranteed almost every year)
- Differentiate Bezier Curve and B-Spline. (5 marks — very common)
- Explain Wireframe and Polygonal Models. (5 marks)

UNIT VI — MULTIMEDIA

6.1 Introduction to Multimedia

Definition

Multimedia refers to the **combination of multiple forms of media/content** — such as **text, images/graphics, audio, video, and animation** — integrated together and delivered/controlled through a computer, to communicate information in a richer, more engaging way than any single medium alone.

Zero-Level Explanation: “Multi” means “many,” and “Media” means “forms of communication.” So Multimedia simply means “many forms of communication combined.” A YouTube video is multimedia — it has video (moving pictures), audio (sound), and often text (captions/titles) — all combined and played together through your computer/phone.

Elements/Components of Multimedia:

1. **Text** – written words, the most basic form of information.
2. **Graphics/Images** – still pictures, drawings, photographs, icons.
3. **Audio** – sound, music, voice narration, sound effects.
4. **Video** – moving images/motion pictures, often combined with audio.
5. **Animation** – simulated movement created by displaying a sequence of images (frames) rapidly.

Characteristics of Multimedia:

- **Multiple media types** combined together.
- Usually **computer-controlled** (interactive or programmable).
- Can be **linear** (plays start to finish, like a movie) or **non-linear/interactive** (user controls navigation, like a website or game — this is called **Hypermedia**, covered later).

Why is Multimedia Important?

- Makes information **more engaging, memorable, and easier to understand** compared to plain text alone.
- Widely used in **education (e-learning), entertainment, business presentations, advertising, and communication.**

Memory Trick: Multimedia elements = “**TIGAV**” – Text, Images, Audio (mixed order for memorability: “Tiny Images Grow Audio Videos”) — or simply remember: “**Text, Graphics, Audio, Video, Animation.**”

6.2 Multimedia Hardware

Definition: Multimedia Hardware refers to the **physical devices/equipment** required to create, capture, process, store, and play back multimedia content.

Categories of Multimedia Hardware:

1. Input Hardware (for capturing multimedia content)

Device	Purpose
Microphone	Captures/records audio (voice, sound)
Camera/Webcam	Captures still images and video
Scanner	Converts printed images/documents into digital format
Graphics Tablet/Digitizer	Allows freehand digital drawing
MIDI Keyboard	Captures/inputs musical notes digitally (Musical Instrument Digital Interface)

2. Processing Hardware

Device	Purpose
CPU	General processing of multimedia application logic
GPU (Graphics Card)	Accelerates rendering of graphics, video, and animation
Sound Card	Processes and converts digital audio signals for playback/recording

3. Storage Hardware

Device	Purpose
Hard Disk / SSD	Stores large multimedia files (video, audio take a lot of space)
CD/DVD/Blu-ray	Optical storage, historically used for distributing movies/software
Pen Drive / Memory Card	Portable storage for multimedia files
Cloud Storage	Online storage for multimedia content

Device	Purpose
	(modern trend)

4. Output Hardware (for playback/display of multimedia content)

Device	Purpose
Monitor/Display Screen	Displays visual multimedia content (images, video)
Speakers/Headphones	Plays audio output
Projector	Displays multimedia content on a large screen (presentations, theatres)
Printer	Produces hard copies of multimedia documents/images

Exam Tip: “Explain Multimedia Hardware with examples” is a **common 5/10 mark question** — organize your answer using the 4 categories above (Input, Processing, Storage, Output) for a clear, well-structured answer.

6.3 Multimedia Software Tools

Definition: Multimedia Software Tools are **application programs** used to create, edit, and manage different types of multimedia content.

Categories of Multimedia Software:

1. Text/Document Editing Software

- Examples: Microsoft Word, Notepad, Google Docs.
- Used to create and format textual content.

2. Image/Graphics Editing Software

- Examples: Adobe Photoshop, CorelDRAW, GIMP, Adobe Illustrator.
- Used to create, edit, and enhance images/graphics.

3. Audio Editing Software

- Examples: Adobe Audition, Audacity, FL Studio.
- Used to record, edit, mix, and enhance audio/sound files.

4. Video Editing Software

- Examples: Adobe Premiere Pro, Final Cut Pro, DaVinci Resolve, Filmora.
- Used to edit, cut, merge, and add effects to video content.

5. Animation Software

- Examples: Adobe Animate, Blender, Autodesk Maya, 3ds Max.
- Used to create 2D/3D animations and special effects.

6. Authoring Tools (Multimedia Integration Software)

- Examples: Adobe Director, Macromedia Authorware, Microsoft PowerPoint.

- **Definition:** Authoring Tools are special software that allow developers to **combine/integrate different multimedia elements (text, audio, video, images, animation) together** into a single interactive application/presentation, often without needing to write extensive programming code.

Comparison Table: Multimedia Software Categories

Category	Purpose	Examples
Text Editing	Create/format text	MS Word, Notepad
Image Editing	Create/edit images	Photoshop, GIMP
Audio Editing	Record/edit sound	Audacity, Adobe Audition
Video Editing	Edit/produce videos	Premiere Pro, Filmora
Animation	Create animations	Blender, Maya
Authoring Tools	Combine all elements into one application	Adobe Director, PowerPoint

Memory Trick: “**TI**AVA-A” – Text, Image, Audio, Video, Animation, Authoring tools.

6.4 Application Areas of Multimedia

Multimedia is used across a wide range of industries and fields:

1. Education (E-Learning)

- Interactive learning modules, educational videos, virtual classrooms, digital textbooks.
- Makes complex topics easier to understand using visuals, animations, and audio explanations.

2. Entertainment

- Movies, video games, music, animated films, streaming platforms (Netflix, YouTube).

3. Business and Corporate

- Presentations, video conferencing, digital marketing/advertisements, product demos, training materials for employees.

4. Advertising and Marketing

- TV commercials, digital banners, social media video ads, interactive product catalogs.

5. Medicine/Healthcare

- Medical imaging, training simulations for surgery, patient education videos, telemedicine.

6. Engineering and Architecture

- 3D modeling and simulation of designs, virtual walkthroughs of buildings before construction.

7. Journalism and Publishing

- Digital newspapers/magazines with embedded video, interactive infographics.

8. Public Information/Government Services

- Information kiosks, public awareness campaigns using video and audio.

9. Virtual Reality (VR) and Augmented Reality (AR)

- Immersive multimedia experiences combining 3D graphics, audio, and interactivity for training, gaming, and simulation.

Table: Application Areas of Multimedia (Quick Revision)

Field	Use of Multimedia
Education	E-learning modules, virtual classrooms
Entertainment	Movies, games, streaming
Business	Presentations, marketing videos
Medicine	Medical imaging, surgery simulation
Engineering	3D design visualization
Journalism	Digital news with video/infographics
Government	Public information kiosks
VR/AR	Immersive training and gaming

Exam Tip: “Explain Application Areas of Multimedia” — a **frequently asked 5/10 mark question**; mention at least 5-6 areas with a one-line explanation each.

6.5 Multimedia Components

This refers back to the fundamental building blocks that make up any multimedia system/presentation:

1. **Text** – Provides the core information/labels; still the most common way to convey precise information.
2. **Graphics/Images** – Add visual interest, illustrate concepts, provide visual data (charts).
3. **Audio** – Adds sound effects, background music, or voice narration, making content more engaging and accessible (especially for visually impaired users).
4. **Video** – Combines moving visuals with audio, extremely effective for storytelling, demonstrations, and tutorials.
5. **Animation** – Simulated motion; useful for explaining processes/concepts (e.g., how an engine works) that would be hard to explain with a still image.

Table: Multimedia Components — File Format Examples

Component	Common File Formats
Text	.txt, .doc, .pdf
Image	.jpg, .png, .gif, .bmp
Audio	.mp3, .wav, .aac
Video	.mp4, .avi, .mkv, .mov

Component	Common File Formats
Animation	.gif, .swf (older Flash), .mp4

Hypermedia

Definition

Hypermedia is an extension of the concept of **Hypertext** (clickable, linked text — like clicking a blue underlined link in a web browser to jump to another page), but generalized to include **all types of multimedia elements (not just text)** — meaning images, audio, and video can also act as clickable **links** that connect to other multimedia content, in a **non-linear (user-controlled)** way.

Zero-Level Explanation: A regular book is “linear” — you read page 1, then 2, then 3, in strict order. A website is “hypermedia” — you can click a picture, which takes you to a video, which has a link to some more text, and so on, in **any order YOU choose**. This non-linear, user-driven navigation through connected multimedia content is Hypermedia.

Key Characteristics of Hypermedia:

- **Non-linear structure** — the user navigates in their own chosen order (unlike a movie, which plays start to finish).
- **Multiple media types linked together** (not just text-to-text links, but text-to-image, image-to-video, etc.)
- Provides **user control and interactivity**.

Hypertext vs Hypermedia — Comparison Table

Basis	Hypertext	Hypermedia
Content type	Only text-based links	Links can include text, image, audio, video
Scope	Narrower (subset)	Broader (superset — includes hypertext concept)
Example	Simple linked text documents (early web pages)	Modern websites, interactive multimedia CDs/apps

Examples of Hypermedia:

- The World Wide Web (WWW) itself — clicking links between web pages containing text, images, video.
- Interactive multimedia encyclopedias (e.g., old Encarta CD-ROMs).
- Interactive museum kiosks.

Memory Trick: “Hypertext = clickable TEXT only; Hypermedia = clickable ANYTHING (text/image/audio/video) — Hypermedia is the bigger, richer version.”

Multimedia Technology

Definition: Multimedia Technology refers to the **combined set of hardware, software, techniques, and standards** used to create, compress, store, transmit, and play back multimedia content effectively.

Key Aspects of Multimedia Technology:

1. Compression Technology

- Multimedia files (especially video/audio) are very large in their raw form — **compression** reduces file size for easier storage/transmission.
- **Lossy Compression:** Reduces file size by permanently discarding some data/quality (e.g., JPEG for images, MP3 for audio) — smaller files, some quality loss.
- **Lossless Compression:** Reduces file size **without** losing any data/quality (e.g., PNG for images, FLAC for audio) — larger files than lossy, but perfect quality retained.

2. Streaming Technology

- Allows multimedia content (especially audio/video) to be **played while still being downloaded/transmitted**, rather than requiring the entire file to be downloaded first — this is how services like YouTube and Netflix work.

3. Multimedia Standards/Formats

- Standardized file formats and codecs (coder-decoder) ensure multimedia content created on one system can be played correctly on another — e.g., MP3 (audio), MP4/H.264 (video), JPEG (images).

4. Virtual Reality (VR) and Augmented Reality (AR) Technology

- **VR:** Creates a completely **simulated, immersive digital environment** that replaces the user's real-world view (using VR headsets).
- **AR: Overlays digital multimedia content on top of the real world** (viewed through a phone camera or AR glasses) — e.g., Pokémon Go, Instagram filters.

Advantages of Multimedia Technology

- Makes information **more accessible, engaging, and easier to understand**.
- Enables **remote learning, telemedicine, virtual meetings**, and global communication.
- Powers modern entertainment and creative industries.

Disadvantages of Multimedia Technology

- Requires **significant hardware resources** (processing power, storage, bandwidth) for high-quality content.
- **Large file sizes** even with compression, especially for high-definition video.
- Risk of **information overload** or distraction if multimedia elements are poorly designed/overused.

Exam Tip: “What is Multimedia Technology? Explain compression and streaming” — commonly asked 5-mark question; mention lossy vs lossless compression with examples.

IMPORTANT POINTS TO REMEMBER (UNIT VI QUICK RECAP)

- **Multimedia** = combination of Text, Graphics, Audio, Video, Animation, controlled via computer.
- **Multimedia Hardware** = Input (mic, camera, scanner) + Processing (CPU, GPU, sound card) + Storage (HDD, SSD, cloud) + Output (monitor, speakers, projector).
- **Multimedia Software** = Text editors, Image editors (Photoshop), Audio editors (Audacity), Video editors (Premiere Pro), Animation tools (Blender), Authoring tools (Director, PowerPoint).
- **Application Areas:** Education, Entertainment, Business, Medicine, Engineering, Journalism, Government, VR/AR.
- **Hypermedia** = non-linear linking of multiple media types (extension of hypertext, which is text-only linking).
- **Multimedia Technology** includes Compression (Lossy vs Lossless), Streaming, Standards/Formats, and VR/AR.

CHAPTER-END QUESTIONS — UNIT VI

20 Short Questions

1. Define Multimedia.
2. Name the five basic elements of Multimedia.
3. What is Multimedia Hardware? Give two examples.
4. What is a Sound Card used for?
5. Name two Image Editing software.
6. Name two Video Editing software.
7. What is an Authoring Tool?
8. Give two application areas of Multimedia.
9. Define Hypertext.
10. Define Hypermedia.
11. Differentiate Hypertext and Hypermedia (one line).
12. What is Lossy Compression?
13. What is Lossless Compression?
14. Give one example each of Lossy and Lossless image formats.
15. What is Streaming Technology?
16. Define Virtual Reality (VR).
17. Define Augmented Reality (AR).
18. Give one example of an AR application.
19. Name any two multimedia input devices.
20. What is MIDI?

15 Long Questions

1. Define Multimedia and explain its elements/components in detail.
2. Explain Multimedia Hardware categories with examples.
3. Explain Multimedia Software Tools with examples in each category.

4. Explain the application areas of Multimedia in detail.
5. Explain Hypermedia and differentiate it from Hypertext.
6. Explain Multimedia Technology — compression, streaming, and standards.
7. Differentiate Lossy and Lossless compression with examples.
8. Explain the role of Authoring Tools in Multimedia development.
9. Explain Virtual Reality and Augmented Reality with examples.
10. Explain the importance of Multimedia in Education (E-Learning).
11. Explain the importance of Multimedia in Business and Marketing.
12. Explain input, processing, storage, and output hardware for multimedia with examples.
13. Explain advantages and disadvantages of Multimedia Technology.
14. Explain common multimedia file formats for each component (text, image, audio, video).
15. Discuss the role of Multimedia in modern communication and entertainment.

10 Viva Questions

1. What does “multi” in Multimedia mean?
2. Give one example of multimedia authoring software.
3. Which device is used to capture audio?
4. Name one lossy audio format.
5. Name one lossless image format.
6. What is the key difference between Hypertext and Hypermedia?
7. What does VR stand for?
8. What does AR stand for?
9. Give one real-life example of AR.
10. Name one streaming platform.

MCQs with Answers

1. Which is NOT a basic element of Multimedia?
(a) Text (b) Audio (c) Spreadsheet (d) Video — **Answer: (c) Spreadsheet**
2. Which software is used for audio editing?
(a) Photoshop (b) Audacity (c) Blender (d) AutoCAD — **Answer: (b) Audacity**
3. MIDI is related to:
(a) Video (b) Music/Audio input (c) Text (d) Animation — **Answer: (b) Music/Audio input**
4. Authoring Tools are used to:
(a) Edit only text (b) Combine multiple media elements into one application (c) Compress files (d) Capture video only — **Answer: (b) Combine multiple media elements into one application**
5. Hypermedia is an extension of:
(a) Hypertext (b) Hardware (c) Compression (d) Streaming — **Answer: (a) Hypertext**
6. JPEG is an example of:

- (a) Lossless compression (b) Lossy compression (c) No compression (d) Streaming format — **Answer: (b) Lossy compression**
7. PNG is an example of:
 (a) Lossy compression (b) Lossless compression (c) Streaming format (d) Authoring tool — **Answer: (b) Lossless compression**
8. Streaming allows content to be:
 (a) Downloaded fully before playing (b) Played while still downloading (c) Only played offline (d) Compressed automatically — **Answer: (b) Played while still downloading**
9. VR stands for:
 (a) Virtual Reality (b) Video Recording (c) Visual Rendering (d) Vector Raster — **Answer: (a) Virtual Reality**
10. Pokemon Go is an example of:
 (a) Virtual Reality (b) Augmented Reality (c) Hypertext (d) Lossless compression — **Answer: (b) Augmented Reality**

Previous-Year Style Questions (BRABU Pattern)

- Define Multimedia and explain its components. (10 marks — very frequently asked)
- Explain Multimedia Hardware and Software with examples. (10 marks — common)
- Explain application areas of Multimedia. (5 marks)
- Differentiate Hypertext and Hypermedia. (5 marks — common)
- Write short notes on: (a) Compression (b) Streaming (c) VR/AR (2 marks each)

MASTER QUESTION BANK & FINAL REVISION SECTION (BCA-402)

A. 100 MOST IMPORTANT QUESTIONS (Unit-wise Index)

All 6 units together contain (per-unit breakdown, matching chapter-end sections): - **Unit I:** 20 Short + 15 Long + 10 Viva + 10 MCQ = 55 questions - **Unit II:** 20 Short + 15 Long + 10 Viva + 10 MCQ = 55 questions - **Unit III:** 20 Short + 15 Long + 10 Viva + 10 MCQ = 55 questions - **Unit IV:** 20 Short + 15 Long + 10 Viva + 10 MCQ = 55 questions - **Unit V:** 20 Short + 15 Long + 10 Viva + 10 MCQ = 55 questions - **Unit VI:** 20 Short + 15 Long + 10 Viva + 10 MCQ = 55 questions

Total: 330 practice questions across the handbook. Below is a curated list of the **TOP 100 most exam-critical questions** (highest repeat probability at BRABU), pulled from across all units:

Top 20 “Must-Solve” Numericals (highest weightage)

1. Solve DDA algorithm for a given line (Unit III).
2. Solve Bresenham’s Line Algorithm for a given line (Unit III).
3. Solve Midpoint Circle Algorithm for a given radius (Unit III).
4. Explain Midpoint Ellipse Algorithm with Region 1 calculation (Unit III).
5. Translate a given point/object by given t_x , t_y (Unit IV).
6. Rotate a given point by a given angle about the origin (Unit IV).

7. Scale a given point/object by given s_x , s_y (Unit IV).
8. Rotate a point about an arbitrary pivot point (3-step method) (Unit IV).
9. Reflect a point about X-axis/Y-axis (Unit IV).
10. Perform a Window-to-Viewport transformation numerical (Unit IV).
11. Apply Cohen-Sutherland region codes to a given line and window (Unit IV).
12. Perspective Projection with COP at origin — solve for projected point (Unit V).
13. Perspective Projection with COP at $C(a,b,c)$ — solve for projected point (Unit V).
14. Solve for a point on a Cubic Bezier Curve at a given t value (Unit V).
15. 3D Translation/Scaling of a given point (Unit V).
16. Sutherland-Hodgman polygon clipping — trace through the 4 boundary cases (Unit IV).
17. Compare DDA vs Bresenham using the same line (theory + why Bresenham is preferred) (Unit III).
18. Compare Perspective vs Parallel projection with a diagram (Unit V).
19. Trace Boundary Fill / Flood Fill algorithm on a small grid (Unit III).
20. Explain and numerically demonstrate transformation composition order-dependence (Unit IV).

Top 20 “Must-Write” Theory/Diagram Questions

21. Explain Computer Graphics System components with diagram (Unit I).
22. Differentiate Interactive and Passive Graphics (Unit I).
23. Explain applications of Computer Graphics (Unit I).
24. Explain CRT working with labeled diagram (Unit II).
25. Differentiate Raster Scan and Random Scan Display (Unit II).
26. Explain Color CRT Monitor (Shadow Mask method) (Unit II).
27. Explain DVST — working, advantages, disadvantages (Unit II).
28. Compare CRT, LCD, LED, Plasma displays (Unit II).
29. Explain Graphics Attributes (line, color, fill, text) (Unit II).
30. Explain 8-way symmetry of a circle with diagram (Unit III).
31. Explain 4-way symmetry of an ellipse (Unit III).
32. Differentiate Boundary Fill and Flood Fill (Unit III).
33. Explain Scan-Line Polygon Fill Algorithm (Unit III).
34. Explain Aliasing and Antialiasing techniques (Unit IV).
35. Explain Dithering with types (Unit IV).
36. Explain Cohen-Sutherland Line Clipping with region-code diagram (Unit IV).
37. Explain Sutherland-Hodgman Polygon Clipping (Unit IV).
38. Differentiate Interior and Exterior Clipping (Unit IV).
39. Explain the 3 methods of Text Clipping (Unit IV).
40. Explain Window and Viewport concepts (Unit IV).
41. Explain Hidden Surface Removal algorithms — Z-Buffer, Painter's, Scan-Line (Unit V).
42. Explain Wireframe Models — importance and demerits (Unit V).
43. Explain Polygonal Models (Unit V).
44. Explain properties of Bezier Curves (Unit V).

45. Differentiate Bezier Curve and B-Spline (local vs global control) (Unit V).
46. Explain types of Parallel Projection — Orthographic and Oblique (Unit V).
47. Define Multimedia and explain its 5 components (Unit VI).
48. Explain Multimedia Hardware categories (Unit VI).
49. Explain Multimedia Software Tools categories (Unit VI).
50. Differentiate Hypertext and Hypermedia (Unit VI).

Remaining 50 High-Yield Short/Medium Questions

51. Define Pixel and Frame Buffer (Unit I).
52. What is a GPU? (Unit I)
53. Explain Persistence in CRT (Unit II).
54. What is Refresh Rate and Flicker? (Unit II)
55. Explain Interlaced vs Non-Interlaced scanning (Unit II).
56. Explain Hard Copy Output Devices — printers and plotters (Unit II).
57. What is Scan Conversion? (Unit III)
58. Time complexity of DDA and Bresenham algorithms (Unit III).
59. Explain Arc Generation Algorithm (Unit III).
60. Advantages/disadvantages of Bresenham's algorithm (Unit III).
61. Explain General Bresenham Algorithm for all octants (Unit III).
62. Explain Geometric vs Coordinate Transformation (Unit IV).
63. Explain Shearing with formula and example (Unit IV).
64. What is Transformation Composition and why does order matter? (Unit IV)
65. Explain Viewport Transformation formula with example (Unit IV).
66. What is trivial accept/reject in Cohen-Sutherland? (Unit IV)
67. Explain 3D Rotation about X, Y, Z axes (Unit V).
68. Explain 3D Viewing pipeline steps (Unit V).
69. What is 3D Clipping and how does it differ from 2D? (Unit V)
70. Explain Back-Face Culling (Unit V).
71. What is a Knot Vector in B-Splines? (Unit V)
72. Explain Cavalier vs Cabinet projection (Unit V).
73. Explain Bezier Surfaces (Unit V).
74. Explain applications of Multimedia in Education (Unit VI).
75. Explain applications of Multimedia in Business (Unit VI).
76. Explain Lossy vs Lossless Compression with examples (Unit VI).
77. Explain Streaming Technology (Unit VI).
78. Explain VR and AR with examples (Unit VI).
79. What is an Authoring Tool? Give examples (Unit VI).
80. Explain Multimedia Technology advantages/disadvantages (Unit VI).
81. Write short note on Light Pen (Unit I).
82. Write short note on Digitizer (Unit I).
83. Write short note on Plasma Panel Display (Unit II).
84. Write short note on TFEL Display (Unit II).

85. Write short note on LED Display (Unit II).
86. Write short note on LCD (Unit II).
87. Write short note on Midpoint Circle Algorithm decision parameter (Unit III).
88. Write short note on Region Codes (Unit IV).
89. Write short note on Convex Hull property (Unit V).
90. Write short note on NURBS (Unit V).
91. Write short note on Painter's Algorithm (Unit V).
92. Write short note on Hypermedia examples (Unit VI).
93. Write short note on MIDI (Unit VI).
94. Differentiate CAD and CAM (Unit I).
95. Explain the role of Frame Buffer in Raster Scan Display (Unit II).
96. Explain why circle algorithms avoid direct equation with square root (Unit III).
97. Explain homogeneous coordinates and why they are used (Unit IV).
98. Explain the 9-region diagram in Cohen-Sutherland algorithm (Unit IV).
99. Explain why a wireframe cube can look ambiguous (Necker Cube effect) (Unit V).
100. Explain the difference between Uniform and Non-Uniform B-Splines (Unit V).

B. 50 ONE-LINERS (RAPID FIRE REVISION)

1. Computer Graphics = creating & manipulating images using a computer.
2. Pixel = smallest addressable point on a screen.
3. Interactive Graphics = two-way, real-time user control (e.g., games).
4. Passive Graphics = one-way, no user control (e.g., printed charts).
5. CRT uses an electron gun + phosphor-coated screen.
6. Persistence = how long phosphor glow remains after the beam is removed.
7. Raster Scan draws the whole screen row by row; used in all modern displays.
8. Random Scan (Vector) draws only the picture's lines directly.
9. Color CRT uses 3 electron guns (R, G, B) + shadow mask.
10. DVST stores the image on-screen — no refresh needed, but no color and no selective erase.
11. Plasma Panel uses ionized gas; TFEL uses direct electroluminescence (no gas).
12. LED = semiconductor pixels emit light directly; energy efficient.
13. LCD uses liquid crystals to control light from a backlight.
14. DDA algorithm uses floating-point increments based on slope.
15. Bresenham's algorithm uses only integer arithmetic — faster and more accurate than DDA.
16. A circle has 8-way symmetry; only 1 octant needs calculation.
17. An ellipse has 4-way symmetry and requires 2 regions (different slope behavior).
18. Boundary Fill stops at a specific border color.
19. Flood Fill replaces all pixels of a specific old interior color.
20. Scan-Line Fill processes a polygon row by row — most efficient for complex polygons.
21. Aliasing = jagged edges due to the pixel grid.
22. Antialiasing smooths jagged edges using techniques like supersampling.

23. Dithering simulates more colors using patterns of fewer available colors.
24. The 5 basic 2D transformations: Translation, Rotation, Scaling, Reflection, Shearing.
25. Homogeneous coordinates add a third "1" value to allow matrix-based transformations.
26. Transformation Composition = combining transformations via matrix multiplication; order matters.
27. To rotate about an arbitrary point: translate to origin → rotate → translate back.
28. Window = part of world coordinates selected; Viewport = screen area where it's displayed.
29. Clipping removes parts of a picture outside the clip window.
30. Cohen-Sutherland uses 4-bit region codes to clip lines (9 regions).
31. Sutherland-Hodgman clips polygons edge-by-edge, boundary-by-boundary.
32. Text Clipping precision order: All-or-None String < All-or-None Character < Exact/Component.
33. 3D transformations use 4×4 matrices (extra z-coordinate).
34. Perspective Projection: lines converge at COP; realistic but distorts size.
35. Parallel Projection: lines stay parallel; preserves true size; used in engineering drawings.
36. Orthographic Projection: projection lines perpendicular to the plane.
37. Oblique Projection: Cavalier = full-scale depth; Cabinet = half-scale depth.
38. Z-Buffer Algorithm is the most common hidden-surface removal method (hardware-supported).
39. Painter's Algorithm draws surfaces back-to-front (farthest first).
40. Wireframe Models show only edges/vertices — fast but not realistic.
41. Polygonal Models add surfaces — realistic but more processing needed.
42. Bezier Curve passes through only its first and last control points.
43. Bezier Curve has global control — moving one point affects the whole curve.
44. B-Spline has local control — moving one point affects only a nearby section.
45. B-Splines require a Knot Vector; Bezier curves do not.
46. Multimedia = combination of Text, Graphics, Audio, Video, Animation.
47. Multimedia Hardware = Input + Processing + Storage + Output devices.
48. Hypertext = clickable text links only; Hypermedia = clickable text/image/audio/video links.
49. Lossy Compression discards data (e.g., JPEG, MP3); Lossless keeps all data (e.g., PNG, FLAC).
50. VR = fully simulated environment; AR = digital content overlaid on the real world.

C. FORMULA SHEET (QUICK REFERENCE)

Line Drawing

DDA: $\text{Steps} = \max(|dx|, |dy|)$; $x_{\text{inc}} = dx/\text{Steps}$; $y_{\text{inc}} = dy/\text{Steps}$

Bresenham Line ($0 < m < 1$):

$p_0 = 2dy - dx$

if $p_k < 0$: $\text{next} = (x_k + 1, y_k)$, $p(k+1) = p_k + 2dy$

else: $\text{next} = (x_k + 1, y_k + 1)$, $p(k+1) = p_k + 2dy - 2dx$

Circle & Ellipse

Midpoint Circle:

```
p0 = 1 - r  
if pk < 0: next=(xk+1,yk), p(k+1)=pk+2xk+1+1  
else:     next=(xk+1,yk-1), p(k+1)=pk+2xk+1+1-2yk+1  
(stop when x>=y; use 8-way symmetry)
```

Midpoint Ellipse (Region 1):

```
p1_0 = ry2 - rx2ry + rx2/4  
(use 4-way symmetry; switch to Region 2 when 2ry2x >= 2rx2y)
```

2D Transformations (Homogeneous Matrix form)

Translation: $x'=x+tx$, $y'=y+ty$

Rotation: $x'=x \cos\theta - y \sin\theta$, $y'=x \sin\theta + y \cos\theta$

Scaling: $x'=x \cdot sx$, $y'=y \cdot sy$

Reflection (X-axis): $x'=x$, $y'=-y$

Shearing (X-dir): $x'=x+shx \cdot y$, $y'=y$

Rotation about arbitrary point (a,b):

1. Translate: (x-a, y-b)
2. Rotate by θ
3. Translate back: add (a,b)

Viewport Transformation

$Sx = (xv_max - xv_min) / (xw_max - xw_min)$

$Sy = (yv_max - yv_min) / (yw_max - yw_min)$

$xv = xv_min + (xw - xw_min) \cdot Sx$

$yv = yv_min + (yw - yw_min) \cdot Sy$

3D Projections

Perspective (COP at origin, plane $z=d$):

$x_p = x \cdot d / z$, $y_p = y \cdot d / z$

Perspective (COP at C(a,b,c), plane $z=0$):

$x_p = a + (x-a) \cdot [(\theta-c)/(z-c)]$

$y_p = b + (y-b) \cdot [(\theta-c)/(z-c)]$

Bezier Curve (Cubic)

$P(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3$, $0 \leq t \leq 1$

D. IMPORTANT COMPARISON TABLES (QUICK RECAP)

Interactive vs Passive Graphics: Interactive = two-way, real-time, user control, high hardware need (games, CAD) | Passive = one-way, no control, cheap (printed graphs, pre-recorded video).

Raster Scan vs Random Scan: Raster = row-by-row, realistic images, cheap, used everywhere today | Random(Vector) = only picture lines, smooth lines only, expensive, old CAD systems.

DDA vs Bresenham: DDA = floating-point, simple, slower, rounding errors | Bresenham = integer-only, faster, more accurate, industry standard.

Boundary Fill vs Flood Fill: Boundary Fill = stops at border color | Flood Fill = replaces old interior color.

Perspective vs Parallel Projection: Perspective = lines converge at COP, realistic, size distorted (games/movies) | Parallel = lines stay parallel, true size preserved (engineering drawings).

Bezier vs B-Spline: Bezier = global control, degree = control points-1, no knot vector | B-Spline = local control, degree independent, needs knot vector.

Hypertext vs Hypermedia: Hypertext = text-only links | Hypermedia = links across text/image/audio/video (broader).

Lossy vs Lossless Compression: Lossy = smaller size, some quality lost (JPEG, MP3) | Lossless = larger size, perfect quality (PNG, FLAC).

E. LAST MINUTE REVISION SHEET (ONE-PAGE CRAM NOTES)

- **Unit I:** CG = creating/manipulating images. System = Input+Processing+Storage+Output+Software. Interactive (2-way, real-time) vs Passive (1-way, fixed). Applications: CAD, Art, Entertainment, Education, Visualization, GUI, CAM, Business.
- **Unit II:** CRT = electron gun+phosphor, needs refresh. Raster (row-by-row, modern) vs Random/Vector (lines only, old). Color CRT = 3 guns+shadow mask. DVST = no refresh but no color/no selective erase. Plasma=gas; TFEL=direct glow; LED=semiconductor; LCD=liquid crystal+backlight. Hard copy = Printers(Dot Matrix/Inkjet/Laser) + Plotters.
- **Unit III:** DDA=float, slope-based. Bresenham=integer, decision parameter ($p_0=2dy-dx$). Circle=8-way symmetry, $p_0=1-r$. Ellipse=4-way symmetry, 2 regions. Fill: Boundary(border color)/Flood(old color)/Scan-Line(row-by-row, best for polygons).
- **Unit IV:** Aliasing=jagged edges; Antialiasing=smoothing (supersampling). Dithering=fake colors via patterns. 5 Transformations: Translate/Rotate/Scale/Reflect/Shear (homogeneous matrices). Composition=matrix multiply, order matters. Window-Viewport mapping. Cohen-Sutherland=line clip (region codes). Sutherland-Hodgman=polygon clip (edge-by-edge). Text clip: String<Character<Exact.
- **Unit V:** 3D uses 4×4 matrices. Perspective=COP convergence, realistic, size distorted. Parallel=Orthographic/Oblique(Cavalier=full,Cabinet=half), size preserved. Hidden Surface: Z-Buffer(fast,common)/Painter's(sort back-to-front)/Scan-Line/Back-face culling. Wireframe=edges only; Polygonal=surfaces added. Bezier=global control, passes through first/last point, convex hull. B-Spline=local control, needs knot vector.
- **Unit VI:** Multimedia=Text+Graphics+Audio+Video+Animation. Hardware=Input/Processing/Storage/Output. Software=Text/Image/Audio/Video/Animation editors + Authoring Tools. Applications=Education,Entertainment,Business,Medicine,Engineering,Journalism,Govt,

VR/AR. Hypermedia=non-linear multi-type linking (bigger than Hypertext).
 Compression=Lossy(JPEG,MP3)/Lossless(PNG,FLAC). Streaming=play while
 downloading. VR=full simulation; AR=overlay on real world.

F. PYQ ANALYSIS / EXPECTED WEIGHTAGE (TYPICAL BRABU BCA-402 PAPER PATTERN)

Unit	Typical Weightage	Most Repeated Question Types
I	10%	CG System components, Interactive vs Passive, Applications
II	15%	CRT working, Raster vs Random Scan, Display technology comparisons
III	25%	DDA/Bresenham/Circle numericals — highest weightage, practice most
IV	20%	2D Transformations, Clipping algorithms (numerical + theory)
V	20%	Projections, Hidden Surface Removal, Bezier/B-Spline
VI	10%	Multimedia components, Hardware/Software, Hypermedia

Golden Rule: Units III, IV, and V contain almost all the **numerical-based** questions, which carry the highest marks and are the most scoring IF practiced by hand. Practice every solved example in this handbook at least twice with pen and paper before the exam.

FINAL EXAM-DAY TIPS

1. For every algorithm question, ALWAYS structure your answer as: **Theory → Formula → Step-by-step Algorithm → Solved Example (table format) → Advantages/Disadvantages/Applications.**
2. Draw diagrams wherever possible (CRT structure, region codes, symmetry diagrams, wireframe cubes) — examiners give separate marks for correct diagrams.
3. For comparison-type questions (Raster vs Random, Bezier vs B-Spline, Perspective vs Parallel), always answer in **table format** — faster to write and easier for examiners to award marks.
4. For transformation numericals, always show the **matrix form** even if you compute using the simple formula — it shows complete understanding.
5. Revise the “Last Minute Revision Sheet” (Section E above) the night before the exam for a fast, complete recall of the entire syllabus.

END OF HANDBOOK